



Student Name: Kenan Kiplimo

Admission Number: BSCCS/2025/67071

Course/Class: Bachelor of Science in Computer science

Unit Code: BIT4138

Unit Name: Advanced Cryptography

Lecturer Name: Mr. Michael Nyoro

Semester/Academic Year: Year 3 Semester 2, May 2026

Selected Technologies:

- ParrotOS Security
- VSCodium
- Python

GitHub: <https://github.com/Kynan51/advanced-cryptography>

Week 1: Cryptography Environment Setup.....	2
Fig 1: Installation of VSCodium.....	3
Fig 2: Installation of Cryptography Libraries.....	3
Fig 3: OpenSSL Installation Test	4
Fig 4: Local Encryption Script Execution	4
Fig 5: GitHub Repository Initialization	5
Student Reflection	5
Week 2: Classical Cryptography and Cipher Design	7
Fig 1: Caesar Cipher Implementation.....	7
Fig 2: Vigenère Cipher Encryption Process	7
Fig 3: Encryption and Decryption Output.....	8
Fig 4: User Input Validation Interface	8
Fig 5: Cipher Testing Results	9
Student Reflection	9
Week 3: Stream Ciphers and Randomness Testing.....	10
Fig 1: LFSR Generator Implementation.....	11
Fig 2: Pseudorandom Sequence Output.....	11
Fig 3: Statistical Randomness Testing.....	11
Fig 4: RC4 Stream Cipher Simulation	12
Fig 5: Encryption Performance Results.....	12
Student Reflection	13
Week 4: Block Cipher Design and AES.....	13
Fig 1: AES Encryption Script	13
Fig 2: Key Generation Process	14
Fig 3: File Encryption Demonstration	14
Fig 4: Decryption Results	15
Fig 5: AES Performance Testing	15
Student Reflection	16
Week 5: Public Key Cryptography (RSA).....	16

Fig 1: RSA Key Pair Generation.....	17
Fig 2: Public Key Encryption Process	17
Fig 3: Private Key Decryption Results	18
Fig 4: Secure Message Transmission.....	18
Fig 5: RSA Testing and Validation	19
Student Reflection	19
Week 6: Hashing and Password Security	20
Fig 1: SHA-256 Hash Generation	21
Fig 2: Password Hashing System	21
Fig 3: Login Authentication Workflow	21
Fig 4: Hash Verification Results	22
Fig 5: Password Security Testing.....	22
Creative Task: Password Strength Analyzer and Leak Simulator	23
Fig 1: Password Entropy and Failure Mode	23
Fig 2: Simulated Breach / Rainbow Table Match.....	23
Fig 3: Secure Cryptographic Isolation	23
Student Reflection	23
Week 7: Digital Signatures and Certificates	24
Fig 1: Digital Signature Generation.....	24
Fig 2: Signature Verification Process	24
Fig 3: Certificate Creation Using OpenSSL.....	24
Fig 4: Secure Document Validation	25
Fig 5: Certificate Testing Results.....	25
Creative Task: Software Update Integrity Guard	25
Fig 1: Cryptographic Packaging and Core Signing Execution	25
Fig 2: Client Verification Success & Deployment Authorization	25
Fig 3: Malicious Interception and Tamper Defiance	26
Student Reflection	26
Week 8: Public Key Infrastructure	26

Part 1: PKI and Digital Certificates	26
Class Activity 1: PKI Component Identification	26
Class Activity 2: Certificate Analysis	26
OpenSSL Certificate Inspection	28
Digital Certificate Security Incident Report.....	29
Part 2: Asymmetric Ciphers and Diffie-Hellman	30
Class Exercise 1: Understanding Asymmetric Cryptography	30
Class Exercise 2: Complexity and Cryptography	30
Practical Task: Implementing Diffie-Hellman Key Exchange	30
Assignment 2: Diffie-Hellman in Modern Internet Security Report	31
Creativity Assignment.....	31
Student Reflection	32

Week 1: Cryptography Environment Setup

Installation and Configuration of Cryptography Development Environment

Fig 1: Installation of VSCodium

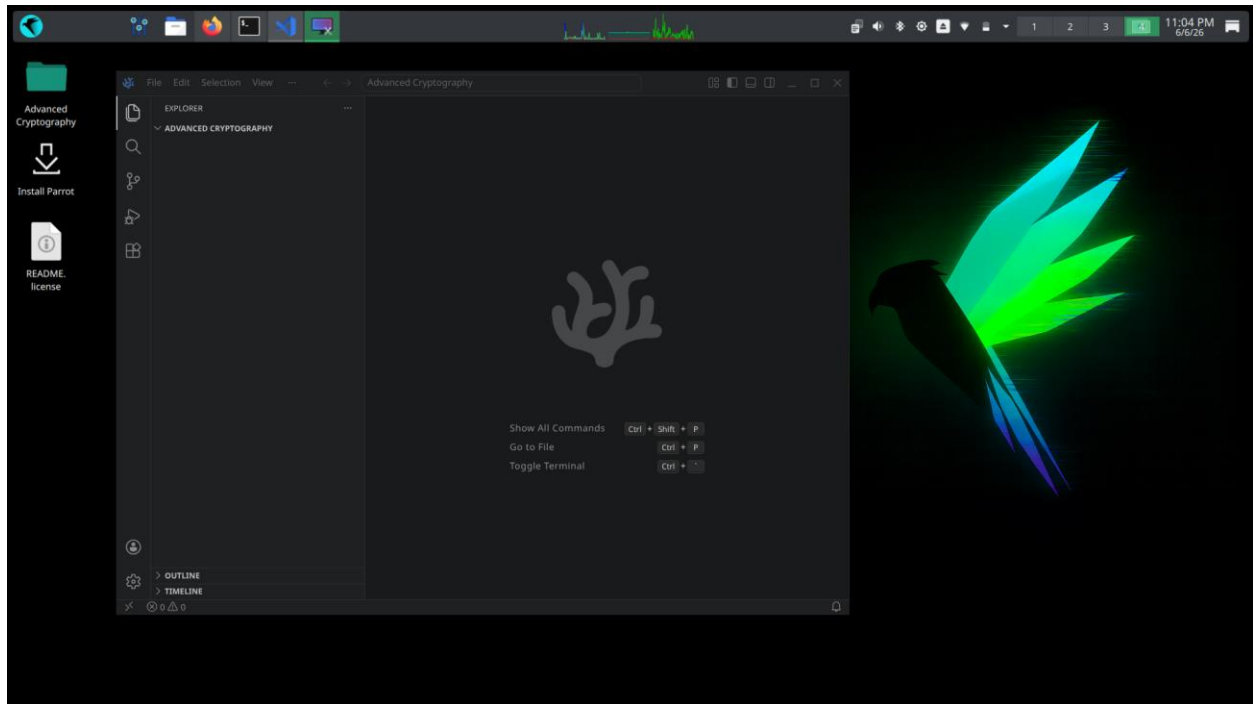
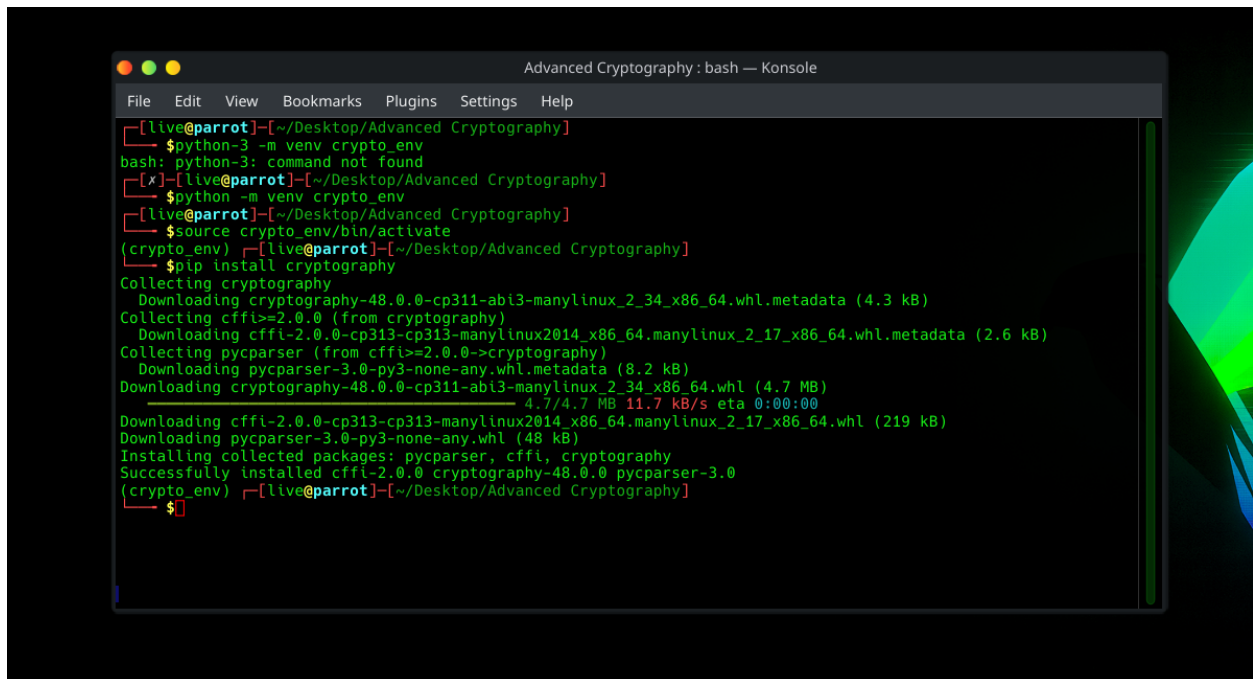
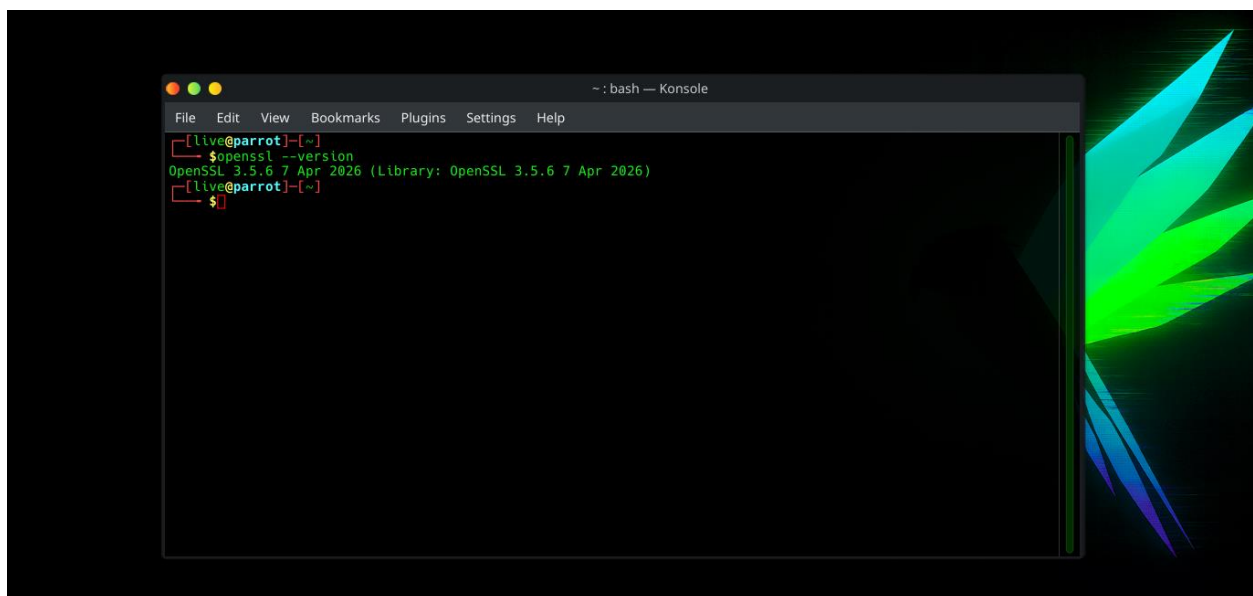


Fig 2: Installation of Cryptography Libraries



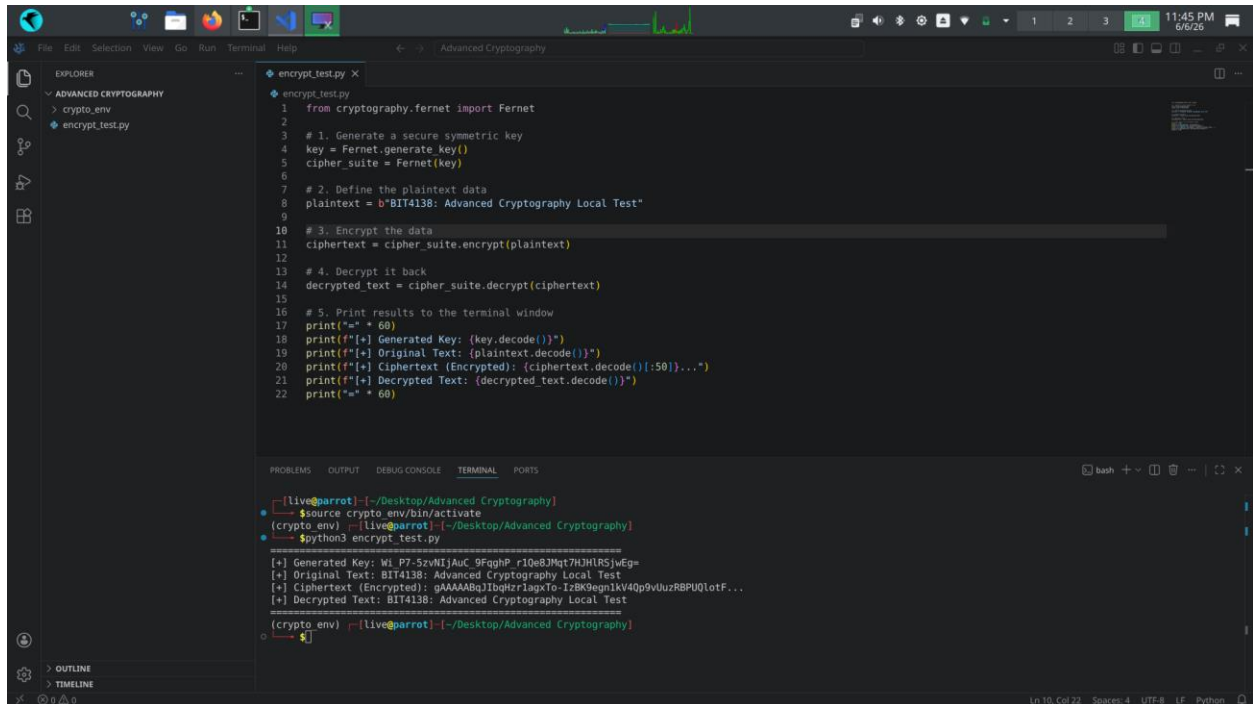
```
Advanced Cryptography : bash — Konsole
File Edit View Bookmarks Plugins Settings Help
[live@parrot] ~/Desktop/Advanced Cryptography
$ python3 -m venv crypto_env
bash: python3: command not found
[x] [live@parrot] ~/Desktop/Advanced Cryptography
$ python -m venv crypto_env
[live@parrot] ~/Desktop/Advanced Cryptography
$ source crypto_env/bin/activate
(crypto_env) [live@parrot] ~/Desktop/Advanced Cryptography
$ pip install cryptography
Collecting cryptography
  Downloading cryptography-48.0.0-cp311-abi3-manylinux_2_34_x86_64.whl.metadata (4.3 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Downloading cffi-2.0.0-cp313-cp313-manylinux2014_x86_64_manylinux_2_17_x86_64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Downloaded cryptography-48.0.0-cp311-abi3-manylinux_2_34_x86_64.whl (4.7 MB)
4.7/4.7 MB 11.7 kB/s eta 0:00:00
Downloaded cffi-2.0.0-cp313-cp313-manylinux2014_x86_64_manylinux_2_17_x86_64.whl (219 kB)
Downloaded pycparser-3.0-py3-none-any.whl (48 kB)
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-48.0.0 pycparser-3.0
(crypto_env) [live@parrot] ~/Desktop/Advanced Cryptography
$
```

Fig 3: OpenSSL Installation Test



```
~ : bash — Konsole
File Edit View Bookmarks Plugins Settings Help
[live@parrot] ~
$ openssl --version
OpenSSL 3.5.6 7 Apr 2026 (Library: OpenSSL 3.5.6 7 Apr 2026)
[live@parrot] ~
$
```

Fig 4: Local Encryption Script Execution

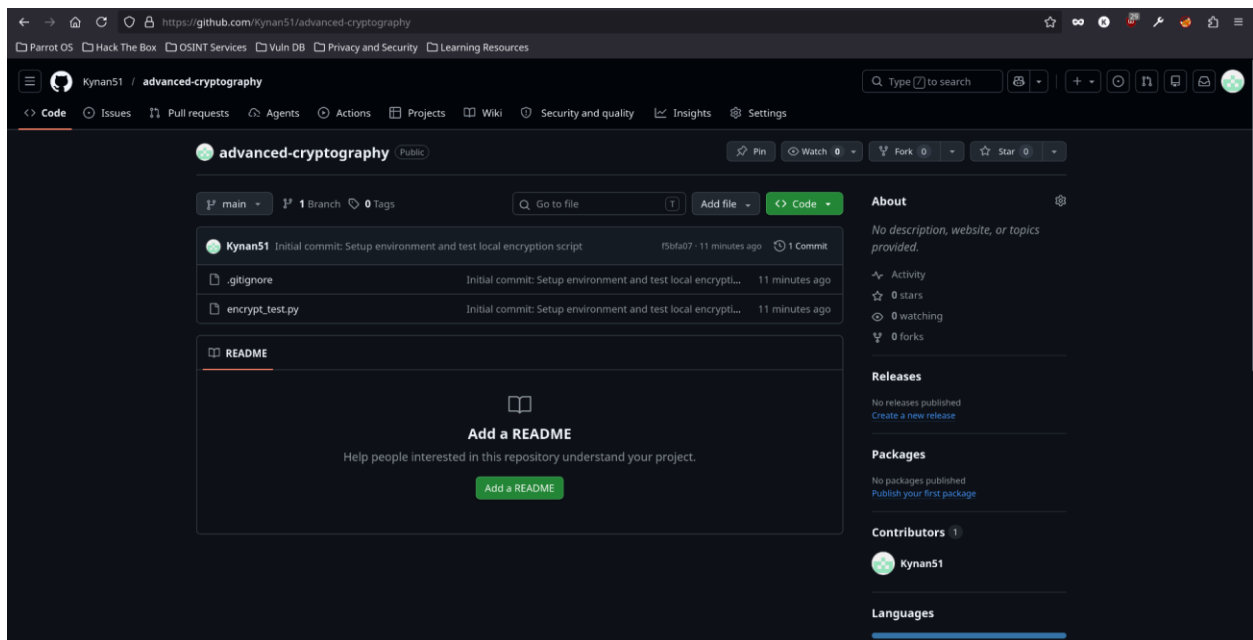


The screenshot shows a VS Code editor with a file explorer on the left containing a folder named 'ADVANCED CRYPTOGRAPHY' with subfolders 'crypto_env' and 'encrypt_test.py'. The main editor window displays the 'encrypt_test.py' script. The script uses the 'cryptography' library's 'Fernet' module to generate a key, encrypt a plaintext string, and then decrypt it back to the original text. The terminal window at the bottom shows the execution of the script, displaying the generated key, the original text, the encrypted ciphertext, and the decrypted text.

```
1 from cryptography.fernet import Fernet
2
3 # 1. Generate a secure symmetric key
4 key = Fernet.generate_key()
5 cipher_suite = Fernet(key)
6
7 # 2. Define the plaintext data
8 plaintext = b"BIT4138: Advanced Cryptography Local Test"
9
10 # 3. Encrypt the data
11 ciphertext = cipher_suite.encrypt(plaintext)
12
13 # 4. Decrypt it back
14 decrypted_text = cipher_suite.decrypt(ciphertext)
15
16 # 5. Print results to the terminal window
17 print(f"Generated Key: {key.decode()}")
18 print(f"Original Text: {plaintext.decode()}")
19 print(f"Ciphertext (Encrypted): {ciphertext.decode()[50:]}...")
20 print(f"Decrypted Text: {decrypted_text.decode()}")
21
22 print(f"")
```

```
[Live@parrot] ~/Desktop/Advanced Cryptography
$source crypto_env/bin/activate
(crypto_env) [Live@parrot] ~/Desktop/Advanced Cryptography
$python3 encrypt_test.py
[+] Generated Key: Wl P7-5zvWtJAuC_9FqgHP_r10e8JMt7HJHRSJwEg=
[+] Original Text: BIT4138: Advanced Cryptography Local Test
[+] Ciphertext (Encrypted): gAAAAABqJIbqHzrIagXo-IzBK9egn1kV40p9V0uzR8PU0lotF...
[+] Decrypted Text: BIT4138: Advanced Cryptography Local Test
(crypto_env) [Live@parrot] ~/Desktop/Advanced Cryptography
$
```

Fig 5: GitHub Repository Initialization



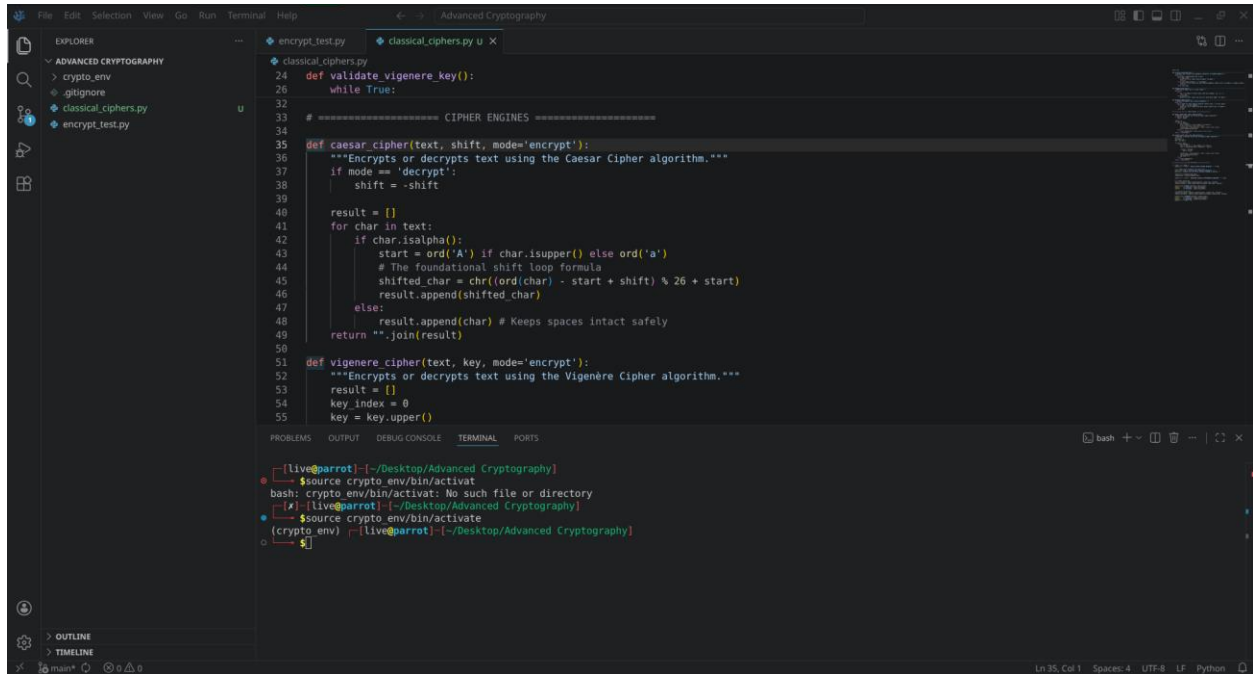
Student Reflection

Configured a portable development environment on a live persistent Parrot OS workstation. Verified the pre-installed OpenSSL architecture and established secure version control via Git. Fig 3 confirms successful installation and configuration of OpenSSL used for generating secure cryptographic keys and certificates.

Week 2: Classical Cryptography and Cipher Design

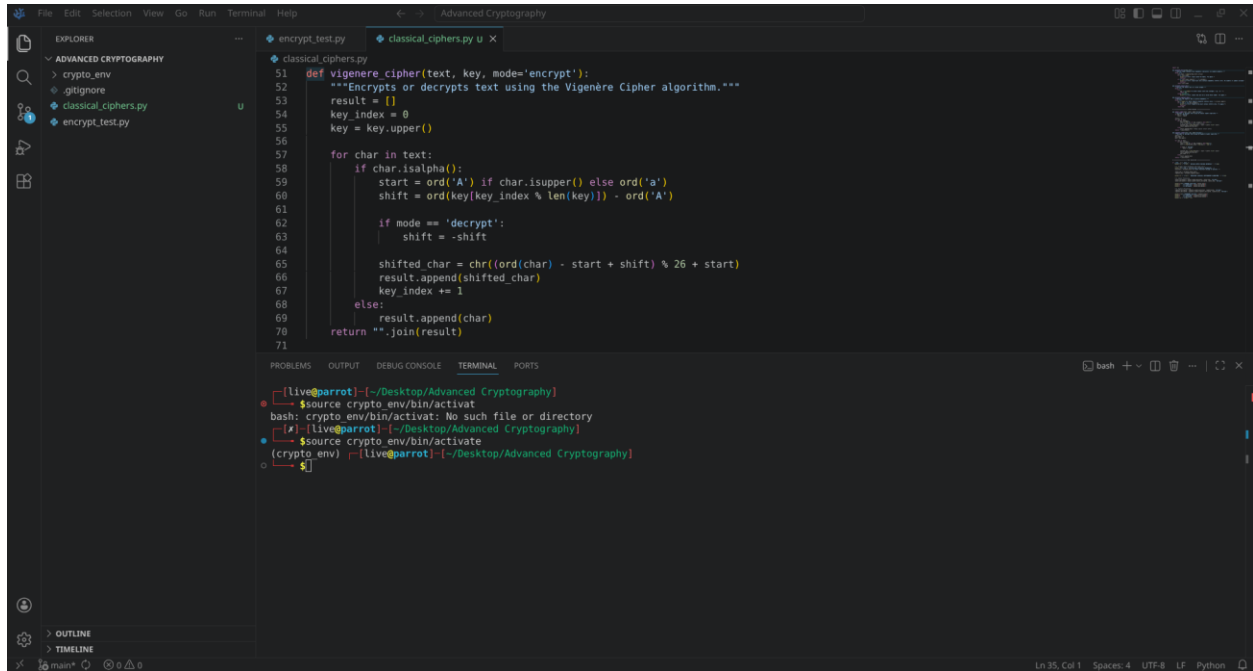
Implementation of Classical Encryption Algorithms

Fig 1: Caesar Cipher Implementation



```
File Edit Selection View Go Run Terminal Help
classical_ciphers.py u
24 def validate_vigenere_key():
25     while True:
26
27 # ===== CIPHER ENGINES =====
28
29 def caesar_cipher(text, shift, mode='encrypt'):
30     """Encrypts or decrypts text using the Caesar Cipher algorithm."""
31     if mode == 'decrypt':
32         shift = -shift
33
34     result = []
35     for char in text:
36         if char.isalpha():
37             start = ord('A') if char.isupper() else ord('a')
38             # The foundational shift loop formula
39             shifted_char = chr((ord(char) - start + shift) % 26 + start)
40             result.append(shifted_char)
41         else:
42             result.append(char) # Keeps spaces intact safely
43     return "".join(result)
44
45 def vigenere_cipher(text, key, mode='encrypt'):
46     """Encrypts or decrypts text using the Vigenere Cipher algorithm."""
47     result = []
48     key_index = 0
49     key = key.upper()
50
51 [live@parrot] ~/Desktop/Advanced_Cryptography
52 $ source crypto_env/bin/activate
53 bash: crypto_env/bin/activate: No such file or directory
54 [x] [live@parrot] ~/Desktop/Advanced_Cryptography
55 $ source crypto_env/bin/activate
56 (crypto_env) [live@parrot] ~/Desktop/Advanced_Cryptography
57 $
```

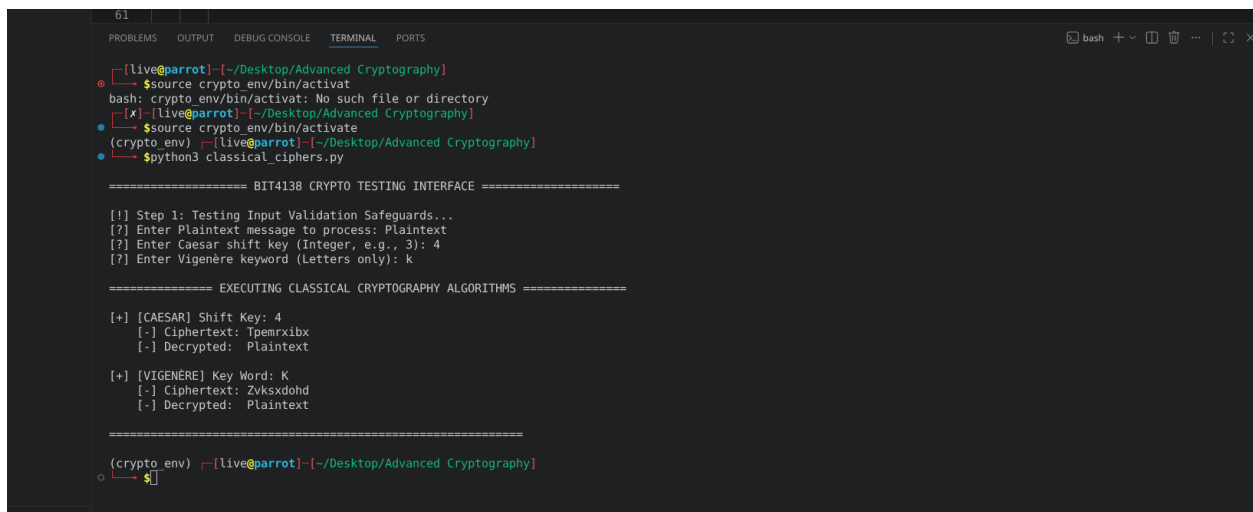
Fig 2: Vigenère Cipher Encryption Process



```
File Edit Selection View Go Run Terminal Help
classical_ciphers.py
51 def vigenere_cipher(text, key, mode='encrypt'):
52     """Encrypts or decrypts text using the Vigenère Cipher algorithm."""
53     result = []
54     key_index = 0
55     key = key.upper()
56
57     for char in text:
58         if char.isalpha():
59             start = ord('A') if char.isupper() else ord('a')
60             shift = ord(key[key_index % len(key)]) - ord('A')
61
62             if mode == 'decrypt':
63                 shift = -shift
64
65             shifted_char = chr((ord(char) - start + shift) % 26 + start)
66             result.append(shifted_char)
67             key_index += 1
68         else:
69             result.append(char)
70     return ''.join(result)
71
```

```
bash
[Live@parrot]~/Desktop/Advanced Cryptography
$source crypto_env/bin/activat
bash: crypto_env/bin/activat: No such file or directory
[Live@parrot]~/Desktop/Advanced Cryptography
$source crypto_env/bin/activate
(crypto_env) [Live@parrot]~/Desktop/Advanced Cryptography
$
```

Fig 3: Encryption and Decryption Output



```
61
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[Live@parrot]~/Desktop/Advanced Cryptography
$source crypto_env/bin/activat
bash: crypto_env/bin/activat: No such file or directory
[Live@parrot]~/Desktop/Advanced Cryptography
$source crypto_env/bin/activate
(crypto_env) [Live@parrot]~/Desktop/Advanced Cryptography
$python3 classical_ciphers.py

===== BIT4138 CRYPTO TESTING INTERFACE =====

[!] Step 1: Testing Input Validation Safeguards...
[?] Enter Plaintext message to process: Plaintext
[?] Enter Caesar shift key (Integer, e.g., 3): 4
[?] Enter Vigenère keyword (Letters only): K

===== EXECUTING CLASSICAL CRYPTOGRAPHY ALGORITHMS =====

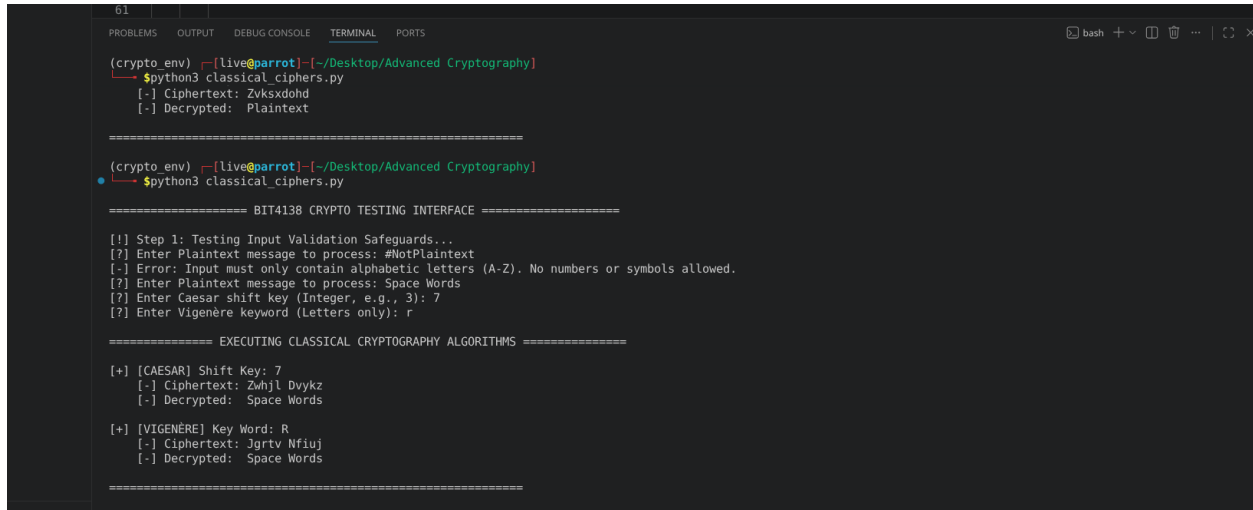
[+] [CAESAR] Shift Key: 4
[-] Ciphertext: Tpemrxibx
[-] Decrypted: Plaintext

[+] [VIGENÈRE] Key Word: K
[-] Ciphertext: Zvksxdohd
[-] Decrypted: Plaintext

=====

(crypto_env) [Live@parrot]~/Desktop/Advanced Cryptography
$
```

Fig 4: User Input Validation Interface



```
(crypto env) [live@parrot] ~/Desktop/Advanced Cryptography
$python3 classical_ciphers.py
[-] Ciphertext: Zvksxdohd
[-] Decrypted: Plaintext

=====

(crypto env) [live@parrot] ~/Desktop/Advanced Cryptography
$python3 classical_ciphers.py

===== BIT4138 CRYPTO TESTING INTERFACE =====

[!] Step 1: Testing Input Validation Safeguards...
[?] Enter Plaintext message to process: #NotPlaintext
[-] Error: Input must only contain alphabetic letters (A-Z). No numbers or symbols allowed.
[?] Enter Plaintext message to process: Space Words
[?] Enter Caesar shift key (Integer, e.g., 3): 7
[?] Enter Vigenère keyword (Letters only): r

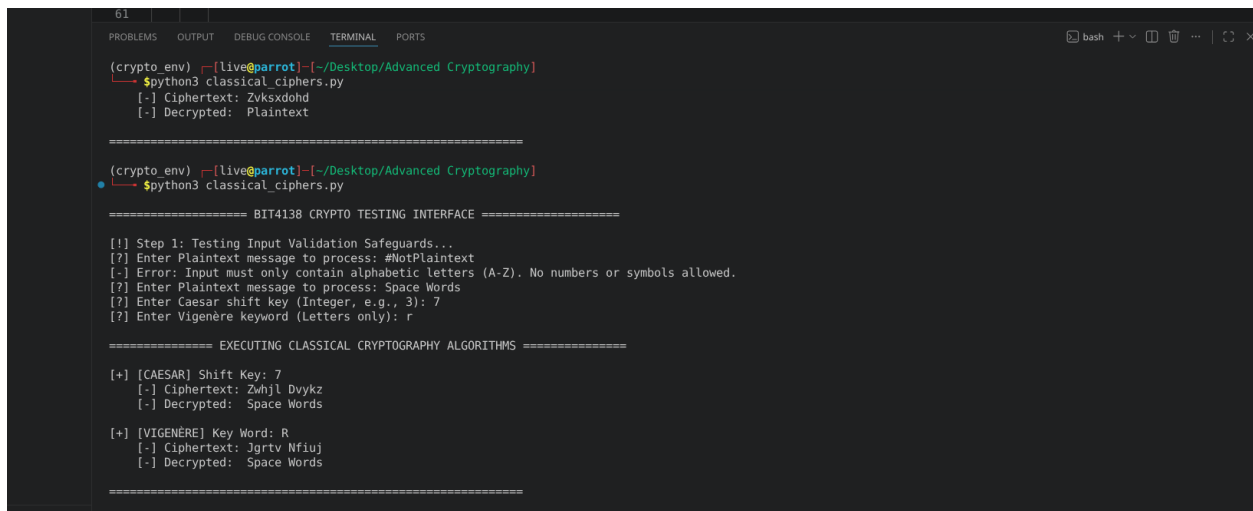
===== EXECUTING CLASSICAL CRYPTOGRAPHY ALGORITHMS =====

[+] [CAESAR] Shift Key: 7
[-] Ciphertext: Zwhjl Dvykz
[-] Decrypted: Space Words

[+] [VIGENÈRE] Key Word: R
[-] Ciphertext: Jgrtv Nfiuj
[-] Decrypted: Space Words

=====
```

Fig 5: Cipher Testing Results



```
(crypto env) [live@parrot] ~/Desktop/Advanced Cryptography
$python3 classical_ciphers.py
[-] Ciphertext: Zvksxdohd
[-] Decrypted: Plaintext

=====

(crypto env) [live@parrot] ~/Desktop/Advanced Cryptography
$python3 classical_ciphers.py

===== BIT4138 CRYPTO TESTING INTERFACE =====

[!] Step 1: Testing Input Validation Safeguards...
[?] Enter Plaintext message to process: #NotPlaintext
[-] Error: Input must only contain alphabetic letters (A-Z). No numbers or symbols allowed.
[?] Enter Plaintext message to process: Space Words
[?] Enter Caesar shift key (Integer, e.g., 3): 7
[?] Enter Vigenère keyword (Letters only): r

===== EXECUTING CLASSICAL CRYPTOGRAPHY ALGORITHMS =====

[+] [CAESAR] Shift Key: 7
[-] Ciphertext: Zwhjl Dvykz
[-] Decrypted: Space Words

[+] [VIGENÈRE] Key Word: R
[-] Ciphertext: Jgrtv Nfiuj
[-] Decrypted: Space Words

=====
```

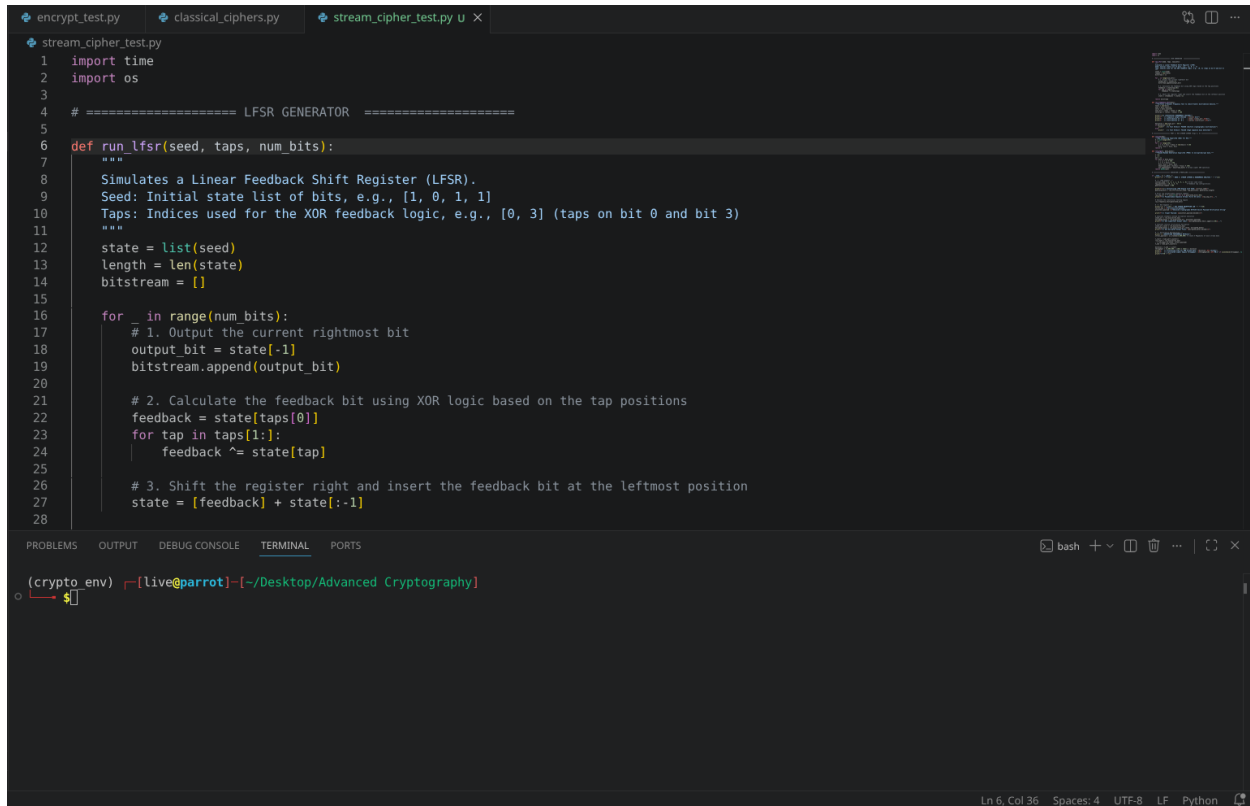
Student Reflection

Implemented functional Caesar and Vigenère polyalphabetic cipher architectures inside Parrot OS to analyze traditional data transformation rules. Developed input sanitization checks to handle structured data constraints defensively. The Vigenère implementation successfully demonstrated how using a variable alphabet rotation counter flattens pattern distributions to mitigate basic statistical frequency cryptanalysis vulnerabilities

Week 3: Stream Ciphers and Randomness Testing

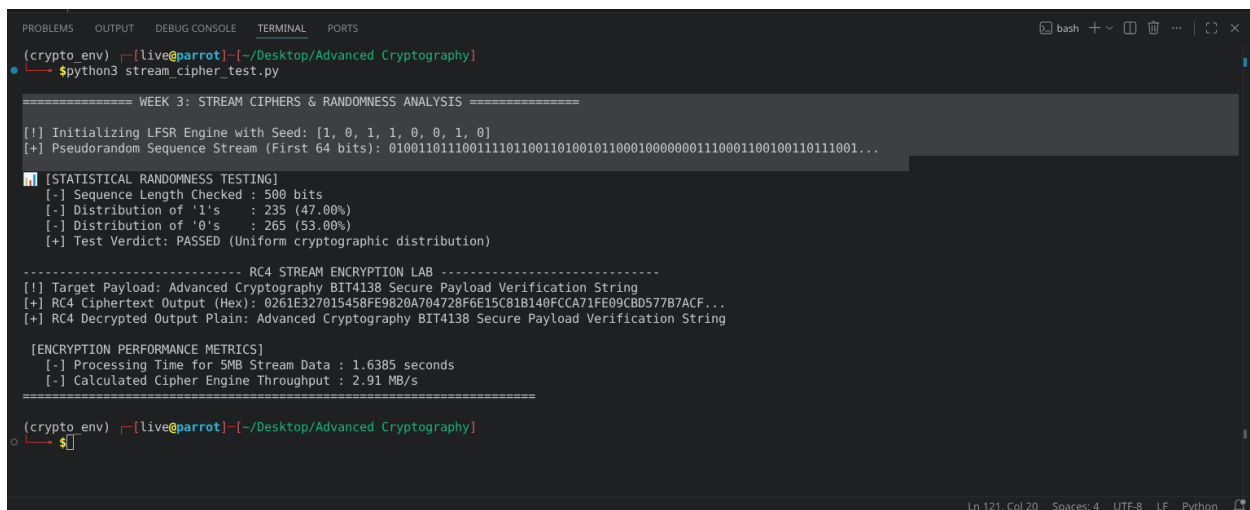
Implementation of Stream Cipher Systems and Random Sequence Analysis

Fig 1: LFSR Generator Implementation



```
stream_cipher_test.py
1 import time
2 import os
3
4 # ===== LFSR GENERATOR =====
5
6 def run_lfsr(seed, taps, num_bits):
7     """
8     Simulates a Linear Feedback Shift Register (LFSR).
9     Seed: Initial state list of bits, e.g., [1, 0, 1, 1]
10    Taps: Indices used for the XOR feedback logic, e.g., [0, 3] (taps on bit 0 and bit 3)
11    """
12    state = list(seed)
13    length = len(state)
14    bitstream = []
15
16    for _ in range(num_bits):
17        # 1. Output the current rightmost bit
18        output_bit = state[-1]
19        bitstream.append(output_bit)
20
21        # 2. Calculate the feedback bit using XOR logic based on the tap positions
22        feedback = state[taps[0]]
23        for tap in taps[1:]:
24            feedback ^= state[tap]
25
26        # 3. Shift the register right and insert the feedback bit at the leftmost position
27        state = [feedback] + state[:-1]
28
```

Fig 2: Pseudorandom Sequence Output



```
(crypto_env) [live@parrot] ~/Desktop/Advanced Cryptography
$ python3 stream_cipher_test.py

===== WEEK 3: STREAM CIPHERS & RANDOMNESS ANALYSIS =====

[!] Initializing LFSR Engine with Seed: [1, 0, 1, 1, 0, 0, 1, 0]
[+] Pseudorandom Sequence Stream (First 64 bits): 010011011100111101100110100100000001110001100100110111001...

[STATISTICAL RANDOMNESS TESTING]
[-] Sequence Length Checked : 500 bits
[-] Distribution of '1's : 235 (47.00%)
[-] Distribution of '0's : 265 (53.00%)
[+] Test Verdict: PASSED (Uniform cryptographic distribution)

===== RC4 STREAM ENCRYPTION LAB =====
[!] Target Payload: Advanced Cryptography BIT4138 Secure Payload Verification String
[+] RC4 Ciphertext Output (Hex): 0261E327015458FE9B20A704728F6E15C81B140FCCA71FE09CB0577B7ACF...
[+] RC4 Decrypted Output Plain: Advanced Cryptography BIT4138 Secure Payload Verification String

[ENCRYPTION PERFORMANCE METRICS]
[-] Processing Time for 5MB Stream Data : 1.6385 seconds
[-] Calculated Cipher Engine Throughput : 2.91 MB/s

(crypto_env) [live@parrot] ~/Desktop/Advanced Cryptography
$
```

Fig 3: Statistical Randomness Testing

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(crypto_env) [live@parrot]~/Desktop/Advanced Cryptography
$python3 stream_cipher_test.py

===== WEEK 3: STREAM CIPHERS & RANDOMNESS ANALYSIS =====

[!] Initializing LFSR Engine with Seed: [1, 0, 1, 1, 0, 0, 1, 0]
[+] Pseudorandom Sequence Stream (First 64 bits): 0100110111001111001101001011000100000001110001100100111001...

[!] [STATISTICAL RANDOMNESS TESTING]
[-] Sequence Length Checked : 500 bits
[-] Distribution of '1's : 235 (47.00%)
[-] Distribution of '0's : 265 (53.00%)
[+] Test Verdict: PASSED (Uniform cryptographic distribution)

----- RC4 STREAM ENCRYPTION LAB -----
[!] Target Payload: Advanced Cryptography BIT4138 Secure Payload Verification String
[+] RC4 Ciphertext Output (Hex): 0261E327015458FE9820A704728F6E15C81B140FCCA71FE09CBD577B7ACF...
[+] RC4 Decrypted Output Plain: Advanced Cryptography BIT4138 Secure Payload Verification String

[ENCRYPTION PERFORMANCE METRICS]
[-] Processing Time for 5MB Stream Data : 1.6385 seconds
[-] Calculated Cipher Engine Throughput : 2.91 MB/s

(crypto_env) [live@parrot]~/Desktop/Advanced Cryptography
$
```

Fig 4: RC4 Stream Cipher Simulation

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(crypto_env) [live@parrot]~/Desktop/Advanced Cryptography
$python3 stream_cipher_test.py

===== WEEK 3: STREAM CIPHERS & RANDOMNESS ANALYSIS =====

[!] Initializing LFSR Engine with Seed: [1, 0, 1, 1, 0, 0, 1, 0]
[+] Pseudorandom Sequence Stream (First 64 bits): 0100110111001111001101001011000100000001110001100100111001...

[!] [STATISTICAL RANDOMNESS TESTING]
[-] Sequence Length Checked : 500 bits
[-] Distribution of '1's : 235 (47.00%)
[-] Distribution of '0's : 265 (53.00%)
[+] Test Verdict: PASSED (Uniform cryptographic distribution)

----- RC4 STREAM ENCRYPTION LAB -----
[!] Target Payload: Advanced Cryptography BIT4138 Secure Payload Verification String
[+] RC4 Ciphertext Output (Hex): 0261E327015458FE9820A704728F6E15C81B140FCCA71FE09CBD577B7ACF...
[+] RC4 Decrypted Output Plain: Advanced Cryptography BIT4138 Secure Payload Verification String

[ENCRYPTION PERFORMANCE METRICS]
[-] Processing Time for 5MB Stream Data : 1.6385 seconds
[-] Calculated Cipher Engine Throughput : 2.91 MB/s

(crypto_env) [live@parrot]~/Desktop/Advanced Cryptography
$
```

Fig 5: Encryption Performance Results

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(crypto_env) [live@parrot]~/Desktop/Advanced Cryptography
$python3 stream_cipher_test.py

===== WEEK 3: STREAM CIPHERS & RANDOMNESS ANALYSIS =====
[!] Initializing LFSR Engine with Seed: [1, 0, 1, 1, 0, 0, 1, 0]
[+] Pseudorandom Sequence Stream (First 64 bits): 010011011100111101100110100100000001110001100100110111001...

[!] [STATISTICAL RANDOMNESS TESTING]
[-] Sequence Length Checked : 500 bits
[-] Distribution of '1's : 235 (47.00%)
[-] Distribution of '0's : 265 (53.00%)
[+] Test Verdict: PASSED (Uniform cryptographic distribution)

----- RC4 STREAM ENCRYPTION LAB -----
[!] Target Payload: Advanced Cryptography BIT4138 Secure Payload Verification String
[+] RC4 Ciphertext Output (Hex): 0261E327015458FE9820A704728F6E15C81B140FCCA71FE09CB0577B7ACF...
[+] RC4 Decrypted Output Plain: Advanced Cryptography BIT4138 Secure Payload Verification String

[!] [ENCRYPTION PERFORMANCE METRICS]
[-] Processing Time for 5MB Stream Data : 1.6385 seconds
[-] Calculated Cipher Engine Throughput : 2.91 MB/s

(crypto_env) [live@parrot]~/Desktop/Advanced Cryptography
$
```

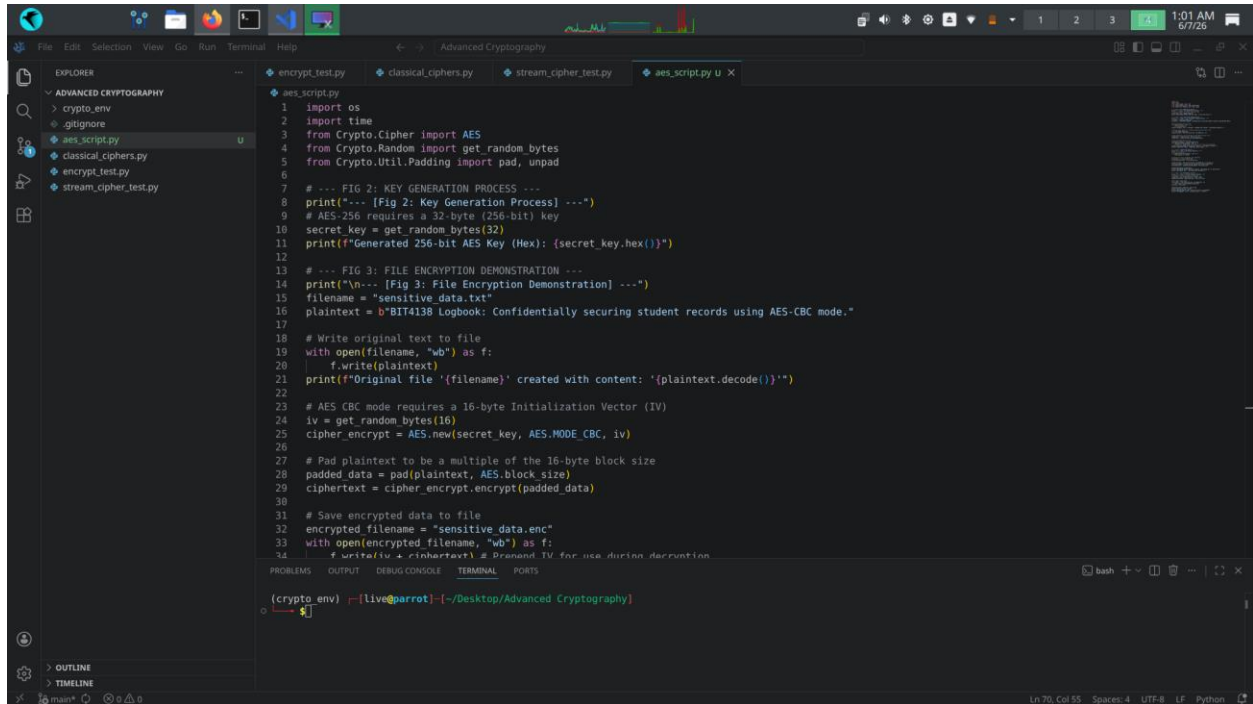
Student Reflection

Simulated an LFSR and RC4 engine to examine continuous byte-level encryption. Cryptographic randomness creates unpredictable key streams to defend against plaintext structural recognition. Executed monobit statistical testing to verify uniform distribution balance, confirming that high-speed throughput and low latency are successfully achieved without introducing mathematical sequence patterns.

Week 4: Block Cipher Design and AES

Advanced Encryption Standard (AES) and Block Cipher Operations

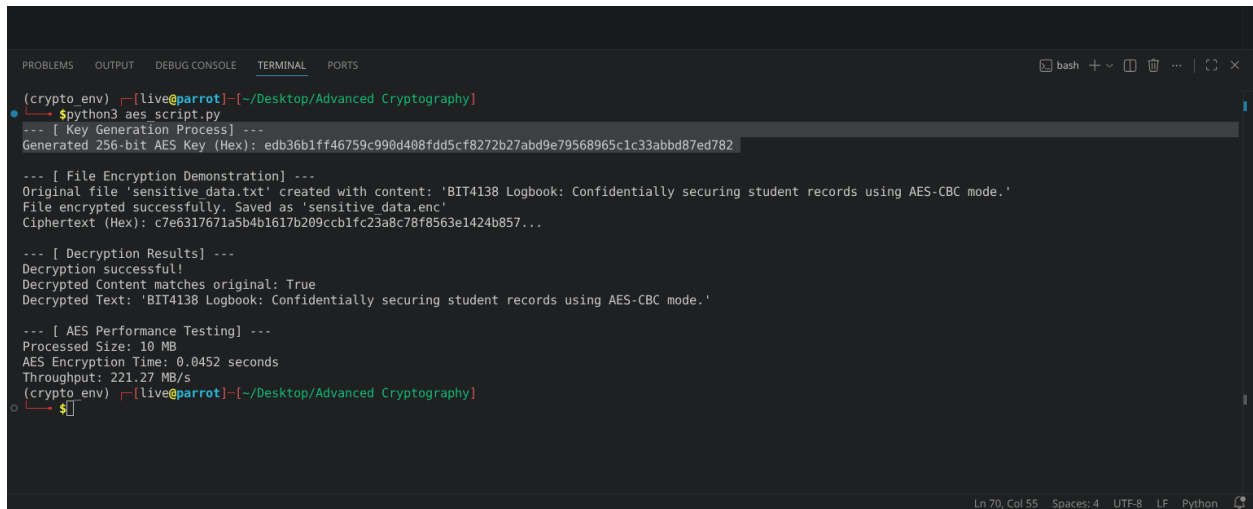
Fig 1: AES Encryption Script



The screenshot shows a code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'Advanced Cryptography' with files like 'crypto_env', '.gitignore', 'aes_script.py', 'classical_ciphers.py', 'encrypt_test.py', and 'stream_cipher_test.py'. The 'aes_script.py' file is open in the editor, showing Python code for AES encryption. The code includes imports for 'os', 'time', 'Crypto.Cipher', 'Crypto.Random', and 'Crypto.Util.Padding'. It defines a function for key generation (Fig 2) and a function for file encryption (Fig 3). The encryption process involves generating a 256-bit key, padding the plaintext, and using AES-CBC mode. The terminal shows the command 'python3 aes_script.py' being executed, and the output displays the generated key and the encryption process.

```
1 import os
2 import time
3 from Crypto.Cipher import AES
4 from Crypto.Random import get_random_bytes
5 from Crypto.Util.Padding import pad, unpad
6
7 # --- FIG 2: KEY GENERATION PROCESS ---
8 print("--- [Fig 2: Key Generation Process] ---")
9 # AES-256 requires a 32-byte (256-bit) key
10 secret_key = get_random_bytes(32)
11 print(f"Generated 256-bit AES Key (Hex): {secret_key.hex()}")
12
13 # --- FIG 3: FILE ENCRYPTION DEMONSTRATION ---
14 print("\n--- [Fig 3: File Encryption Demonstration] ---")
15 filename = "sensitive_data.txt"
16 plaintext = b"BIT4138 Logbook: Confidentially securing student records using AES-CBC mode."
17
18 # Write original text to file
19 with open(filename, "wb") as f:
20     f.write(plaintext)
21 print(f"Original file '{filename}' created with content: '{plaintext.decode()}'")
22
23 # AES CBC mode requires a 16-byte Initialization Vector (IV)
24 iv = get_random_bytes(16)
25 cipher_encrypt = AES.new(secret_key, AES.MODE_CBC, iv)
26
27 # Pad plaintext to be a multiple of the 16-byte block size
28 padded_data = pad(plaintext, AES.block_size)
29 ciphertext = cipher_encrypt.encrypt(padded_data)
30
31 # Save encrypted data to file
32 encrypted_filename = "sensitive_data.enc"
33 with open(encrypted_filename, "wb") as f:
34     f.write(iv + ciphertext) # Prepend IV for use during decryption
```

Fig 2: Key Generation Process



The screenshot shows a terminal window with the output of the 'python3 aes_script.py' command. The output displays the generated 256-bit AES key (Hex: edb36b1ff46759c990d408fdd5cf8272b27abd9e79568965c1c33abbd87ed782), the file encryption process, and the decryption results. The decryption is successful, and the decrypted text matches the original: 'BIT4138 Logbook: Confidentially securing student records using AES-CBC mode.' The terminal also shows performance testing results: 'Processed Size: 10 MB', 'AES Encryption Time: 0.0452 seconds', and 'Throughput: 221.27 MB/s'.

```
(crypto_env) [live@parrot] ~/Desktop/Advanced Cryptography
$ python3 aes_script.py
--- [ Key Generation Process ] ---
Generated 256-bit AES Key (Hex): edb36b1ff46759c990d408fdd5cf8272b27abd9e79568965c1c33abbd87ed782

--- [ File Encryption Demonstration ] ---
Original file 'sensitive_data.txt' created with content: 'BIT4138 Logbook: Confidentially securing student records using AES-CBC mode.'
File encrypted successfully. Saved as 'sensitive_data.enc'
Ciphertext (Hex): c7e6317671a5b4b167b209cbb1fc23a8c78f8563e1424b857...

--- [ Decryption Results ] ---
Decryption successful!
Decrypted Content matches original: True
Decrypted Text: 'BIT4138 Logbook: Confidentially securing student records using AES-CBC mode.'

--- [ AES Performance Testing ] ---
Processed Size: 10 MB
AES Encryption Time: 0.0452 seconds
Throughput: 221.27 MB/s
(crypto_env) [live@parrot] ~/Desktop/Advanced Cryptography
$
```


Fig 3: File Encryption Demonstration

```
56 # --- AES PERFORMANCE TESTING ---
57 print("\n--- [ AES Performance Testing] ---")
58 # Benchmark encrypting 10MB of dummy data
59 large_data = get_random_bytes(10 * 1024 * 1024)
60 padded_large_data = pad(large_data, AES.block_size)
61
62 start_time = time.time()
63 test_cipher = AES.new(secret_key, AES.MODE_CBC, iv)
64 _ = test_cipher.encrypt(padded_large_data)
65 end_time = time.time()
66
67 execution_time = end_time - start_time
68 print(f"Processed Size: 10 MB")
69 print(f"AES Encryption Time: {execution_time:.4f} seconds")
70 print(f"Throughput: {(10 / execution_time):.2f} MB/s")
```

```
(crypto.env) [live@parrot:~/Desktop/Advanced Cryptography]
$python3 aes_script.py
--- [ Key Generation Process] ---
Generated 256-bit AES Key (Hex): edb36b1ff46759c990d408fdd5cf8272b27abd9e79568965c1c3abbd87ed782

--- [ File Encryption Demonstration] ---
Original file 'sensitive_data.txt' created with content: 'BIT4138 Logbook: Confidentially securing student records using AES-CBC mode.'
File encrypted successfully. Saved as 'sensitive_data.enc'
Ciphertext (Hex): c7e6317071a5b4b17b289cbb1fc23a8c78f8563e1424b857...

--- [ Decryption Results] ---
Decryption successful!
Decrypted Content matches original: True
Decrypted Text: 'BIT4138 Logbook: Confidentially securing student records using AES-CBC mode.'

--- [ AES Performance Testing] ---
Processed Size: 10 MB
AES Encryption Time: 0.0452 seconds
Throughput: 221.27 MB/s
(crypto.env) [live@parrot:~/Desktop/Advanced Cryptography]
$
```

Fig 4: Decryption Results

```
56 # --- AES PERFORMANCE TESTING ---
57 print("\n--- [ AES Performance Testing] ---")
58 # Benchmark encrypting 10MB of dummy data
59 large_data = get_random_bytes(10 * 1024 * 1024)
60 padded_large_data = pad(large_data, AES.block_size)
61
62 start_time = time.time()
63 test_cipher = AES.new(secret_key, AES.MODE_CBC, iv)
64 _ = test_cipher.encrypt(padded_large_data)
65 end_time = time.time()
66
67 execution_time = end_time - start_time
68 print(f"Processed Size: 10 MB")
69 print(f"AES Encryption Time: {execution_time:.4f} seconds")
70 print(f"Throughput: {(10 / execution_time):.2f} MB/s")
```

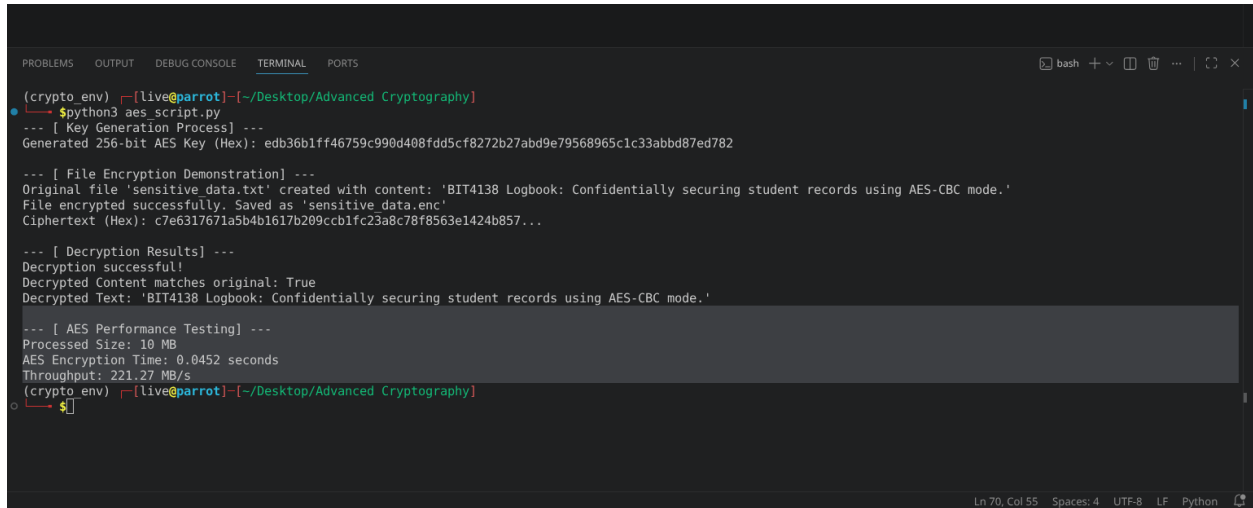
```
(crypto.env) [live@parrot:~/Desktop/Advanced Cryptography]
$python3 aes_script.py
--- [ Key Generation Process] ---
Generated 256-bit AES Key (Hex): edb36b1ff46759c990d408fdd5cf8272b27abd9e79568965c1c3abbd87ed782

--- [ File Encryption Demonstration] ---
Original file 'sensitive_data.txt' created with content: 'BIT4138 Logbook: Confidentially securing student records using AES-CBC mode.'
File encrypted successfully. Saved as 'sensitive_data.enc'
Ciphertext (Hex): c7e6317071a5b4b17b289cbb1fc23a8c78f8563e1424b857...

--- [ Decryption Results] ---
Decryption successful!
Decrypted Content matches original: True
Decrypted Text: 'BIT4138 Logbook: Confidentially securing student records using AES-CBC mode.'

--- [ AES Performance Testing] ---
Processed Size: 10 MB
AES Encryption Time: 0.0452 seconds
Throughput: 221.27 MB/s
(crypto.env) [live@parrot:~/Desktop/Advanced Cryptography]
$
```

Fig 5: AES Performance Testing

A screenshot of a terminal window with a dark background. The terminal shows the execution of a Python script named 'aes_script.py'. The output includes key generation, file encryption, decryption, and performance metrics. The performance section shows a processed size of 10 MB, an encryption time of 0.0452 seconds, and a throughput of 221.27 MB/s. The terminal window has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The status bar at the bottom indicates 'Ln 70, Col 55', 'Spaces: 4', 'UTF-8', 'LF', and 'Python'.

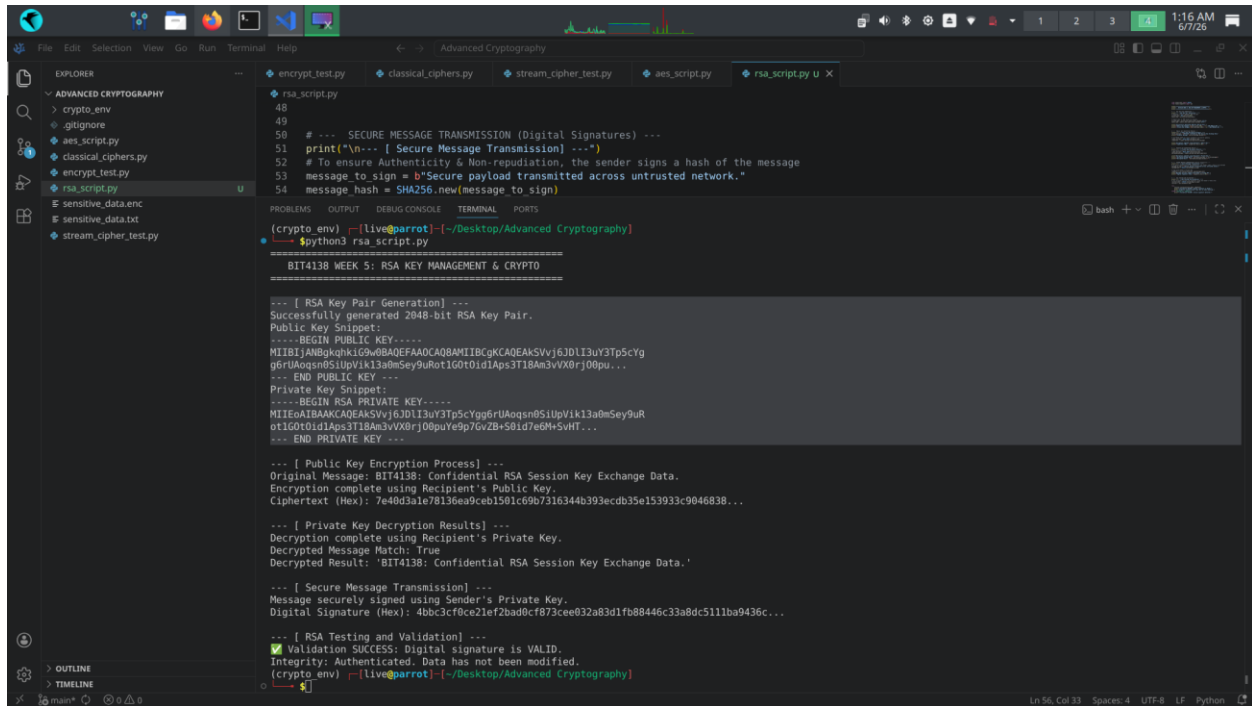
Student Reflection

Implemented AES-256 in CBC mode, mastering its block-by-block diffusion workflow. Correctly padding data and securely handling the initialization vector (IV) was challenging. Secure key management proved vital, as compromised keys invalidate the entire encryption/decryption integrity. Performance testing highlighted AES's exceptional, hardware-optimized computational efficiency with large files.

Week 5: Public Key Cryptography (RSA)

Implementation of RSA Encryption and Key Management

Fig 1: RSA Key Pair Generation



The screenshot displays a code editor with a file explorer on the left and a terminal window at the bottom. The file explorer shows a project named 'ADVANCED CRYPTOGRAPHY' with files like 'crypto_env', 'gitignore', 'aes_script.py', 'classical_ciphers.py', 'encrypt_test.py', and 'rsa_script.py'. The 'rsa_script.py' file is open in the editor, showing Python code for RSA key pair generation and message encryption/decryption. The terminal window shows the output of running 'python3 rsa_script.py' in a 'live@parrot' environment. The output includes the generation of a 2048-bit RSA key pair, the encryption of a message using the public key, and the decryption of the ciphertext using the private key. The decrypted message is 'BIT4138: Confidential RSA Session Key Exchange Data.' The terminal also shows the message being signed and the signature being verified.

```
48
49
50 # --- SECURE MESSAGE TRANSMISSION (Digital Signatures) ---
51 print("\n--- [ Secure Message Transmission] ---")
52 # To ensure Authenticity & Non-repudiation, the sender signs a hash of the message
53 message_to_sign = b"Secure payload transmitted across untrusted network."
54 message_hash = SHA256.new(message_to_sign)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(crypto_env) ~[live@parrot]:~/Desktop/Advanced Cryptography
$python3 rsa_script.py

=====
BIT4138 WEEK 5: RSA KEY MANAGEMENT & CRYPTO
=====

--- [ RSA Key Pair Generation] ---
Successfully generated 2048-bit RSA Key Pair.
Public Key Snippet:
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQFAOACQ8AMIIBgKCAQEAkSVvj6JDL13uY3Tp5CYg
g6rUaQsn85IupVik13a0mSey9uRot1G0t0id1Aps3T18Am3vVXBrj00pu...
--- END PUBLIC KEY ---
Private Key Snippet:
-----BEGIN RSA PRIVATE KEY-----
MIIEoA1BAKCAQEAkSVvj6JDL13uY3Tp5CYgg6rUaQsn85IupVik13a0mSey9uR
ot1G0t0id1Aps3T18Am3vVXBrj00puYe9p7GvZB+S01d7e6H+SVHT...
--- END PRIVATE KEY ---

--- [ Public Key Encryption Process] ---
Original Message: BIT4138: Confidential RSA Session Key Exchange Data.
Encryption complete using Recipient's Public Key.
Ciphertext (Hex): 7e40d3a1e78136ea9ceb1501c69b7516344b393ecdb35e153933c9046838...

--- [ Private Key Decryption Results] ---
Decryption complete using Recipient's Private Key.
Decrypted Message Match: True
Decrypted Result: 'BIT4138: Confidential RSA Session Key Exchange Data.'

--- [ Secure Message Transmission] ---
Message securely signed using Sender's Private Key.
Digital Signature (Hex): 4bb3cf0ce21ef2bad0cf873cee032a83d1fb88446c33a8dc5111ba9436c...

--- [ RSA Testing and Validation] ---
[✓] Validation SUCCESS: Digital signature is VALID.
Integrity: Authenticated. Data has not been modified.
(crypto_env) ~[live@parrot]:~/Desktop/Advanced Cryptography
$
```

Fig 2: Public Key Encryption Process

```
message_new = sha256.new(message_to_sign)

(crypto env) [live@parrot] ~/Desktop/Advanced Cryptography
$python3 rsa_script.py

=====
BIT4138 WEEK 5: RSA KEY MANAGEMENT & CRYPTO
=====

--- [ RSA Key Pair Generation] ---
Successfully generated 2048-bit RSA Key Pair.
Public Key Snippet:
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAKSVvj6JDLI3uY3Tp5cYg
g6rUAoqsn0SiUpVik13a0mSey9uRot1G0t0id1Aps3T18Am3vVX0rj00pu...
--- END PUBLIC KEY ---
Private Key Snippet:
-----BEGIN RSA PRIVATE KEY-----
MIIEoAIBAAKCAQEAKSVvj6JDLI3uY3Tp5cYgg6rUAoqsn0SiUpVik13a0mSey9uR
ot1G0t0id1Aps3T18Am3vVX0rj00puYe9p7GvZB+S0id7e6M+5vHT...
--- END PRIVATE KEY ---

--- [ Public Key Encryption Process] ---
Original Message: BIT4138: Confidential RSA Session Key Exchange Data.
Encryption complete using Recipient's Public Key.
Ciphertext (Hex): 7e40d3a1e78136ea9ceb1501c69b7316344b393ecdb35e153933c9046838...

--- [ Private Key Decryption Results] ---
Decryption complete using Recipient's Private Key.
Decrypted Message Match: True
Decrypted Result: 'BIT4138: Confidential RSA Session Key Exchange Data.'

--- [ Secure Message Transmission] ---
Message securely signed using Sender's Private Key.
Digital Signature (Hex): 4bbc3cf0ce21ef2bad0cf873cee032a83d1fb88446c33a8dc511ba9436c...

--- [ RSA Testing and Validation] ---
✓ Validation SUCCESS: Digital signature is VALID.
Integrity: Authenticated. Data has not been modified.
(crypto env) [live@parrot] ~/Desktop/Advanced Cryptography
$
```

Fig 3: Private Key Decryption Results

```
message_new = sha256.new(message_to_sign)

(crypto env) [live@parrot] ~/Desktop/Advanced Cryptography
$python3 rsa_script.py

=====
BIT4138 WEEK 5: RSA KEY MANAGEMENT & CRYPTO
=====

--- [ RSA Key Pair Generation] ---
Successfully generated 2048-bit RSA Key Pair.
Public Key Snippet:
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAKSVvj6JDLI3uY3Tp5cYg
g6rUAoqsn0SiUpVik13a0mSey9uRot1G0t0id1Aps3T18Am3vVX0rj00pu...
--- END PUBLIC KEY ---
Private Key Snippet:
-----BEGIN RSA PRIVATE KEY-----
MIIEoAIBAAKCAQEAKSVvj6JDLI3uY3Tp5cYgg6rUAoqsn0SiUpVik13a0mSey9uR
ot1G0t0id1Aps3T18Am3vVX0rj00puYe9p7GvZB+S0id7e6M+5vHT...
--- END PRIVATE KEY ---

--- [ Public Key Encryption Process] ---
Original Message: BIT4138: Confidential RSA Session Key Exchange Data.
Encryption complete using Recipient's Public Key.
Ciphertext (Hex): 7e40d3a1e78136ea9ceb1501c69b7316344b393ecdb35e153933c9046838...

--- [ Private Key Decryption Results] ---
Decryption complete using Recipient's Private Key.
Decrypted Message Match: True
Decrypted Result: 'BIT4138: Confidential RSA Session Key Exchange Data.'

--- [ Secure Message Transmission] ---
Message securely signed using Sender's Private Key.
Digital Signature (Hex): 4bbc3cf0ce21ef2bad0cf873cee032a83d1fb88446c33a8dc511ba9436c...

--- [ RSA Testing and Validation] ---
✓ Validation SUCCESS: Digital signature is VALID.
Integrity: Authenticated. Data has not been modified.
(crypto env) [live@parrot] ~/Desktop/Advanced Cryptography
$
```

Fig 4: Secure Message Transmission

```
message_id: 5142307 new message to sign
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(crypto env) [live@parrot] ~/Desktop/Advanced Cryptography
$python3 rsa_script.py
=====
BIT4138 WEEK 5: RSA KEY MANAGEMENT & CRYPTO
=====

--- [ RSA Key Pair Generation] ---
Successfully generated 2048-bit RSA Key Pair.
Public Key Snippet:
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAKSVvj6JDLI3uY3Tp5cYg
g6rUAoqsn0SiUpVik13a0mSey9uRot1G0t0id1Aps3T18Am3vVX0rj00pu...
--- END PUBLIC KEY ---
Private Key Snippet:
-----BEGIN RSA PRIVATE KEY-----
MIIEoAIBAAKCAQEAkSVvj6JDLI3uY3Tp5cYgg6rUAoqsn0SiUpVik13a0mSey9uR
ot1G0t0id1Aps3T18Am3vVX0rj00puYe9p7GvZB+S0id7e6M+5vHT...
--- END PRIVATE KEY ---

--- [ Public Key Encryption Process] ---
Original Message: BIT4138: Confidential RSA Session Key Exchange Data.
Encryption complete using Recipient's Public Key.
Ciphertext (Hex): 7e40d3a1e78136ea9ceb1501c69b7316344b393ecdb35e153933c9046838...

--- [ Private Key Decryption Results] ---
Decryption complete using Recipient's Private Key.
Decrypted Message Match: True
Decrypted Result: 'BIT4138: Confidential RSA Session Key Exchange Data.'

--- [ Secure Message Transmission] ---
Message securely signed using Sender's Private Key.
Digital Signature (Hex): 4bbc3cf0ce21ef2bad0cf873cee032a83d1fb88446c33a8dc511ba9436c...

--- [ RSA Testing and Validation] ---
✓ Validation SUCCESS: Digital signature is VALID.
Integrity: Authenticated. Data has not been modified.
(crypto env) [live@parrot] ~/Desktop/Advanced Cryptography
$
```

Fig 5: RSA Testing and Validation

```
message_id: 5142307 new message to sign
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(crypto env) [live@parrot] ~/Desktop/Advanced Cryptography
$python3 rsa_script.py
=====
BIT4138 WEEK 5: RSA KEY MANAGEMENT & CRYPTO
=====

--- [ RSA Key Pair Generation] ---
Successfully generated 2048-bit RSA Key Pair.
Public Key Snippet:
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAKSVvj6JDLI3uY3Tp5cYg
g6rUAoqsn0SiUpVik13a0mSey9uRot1G0t0id1Aps3T18Am3vVX0rj00pu...
--- END PUBLIC KEY ---
Private Key Snippet:
-----BEGIN RSA PRIVATE KEY-----
MIIEoAIBAAKCAQEAkSVvj6JDLI3uY3Tp5cYgg6rUAoqsn0SiUpVik13a0mSey9uR
ot1G0t0id1Aps3T18Am3vVX0rj00puYe9p7GvZB+S0id7e6M+5vHT...
--- END PRIVATE KEY ---

--- [ Public Key Encryption Process] ---
Original Message: BIT4138: Confidential RSA Session Key Exchange Data.
Encryption complete using Recipient's Public Key.
Ciphertext (Hex): 7e40d3a1e78136ea9ceb1501c69b7316344b393ecdb35e153933c9046838...

--- [ Private Key Decryption Results] ---
Decryption complete using Recipient's Private Key.
Decrypted Message Match: True
Decrypted Result: 'BIT4138: Confidential RSA Session Key Exchange Data.'

--- [ Secure Message Transmission] ---
Message securely signed using Sender's Private Key.
Digital Signature (Hex): 4bbc3cf0ce21ef2bad0cf873cee032a83d1fb88446c33a8dc511ba9436c...

--- [ RSA Testing and Validation] ---
✓ Validation SUCCESS: Digital signature is VALID.
Integrity: Authenticated. Data has not been modified.
(crypto env) [live@parrot] ~/Desktop/Advanced Cryptography
$
```

Student Reflection

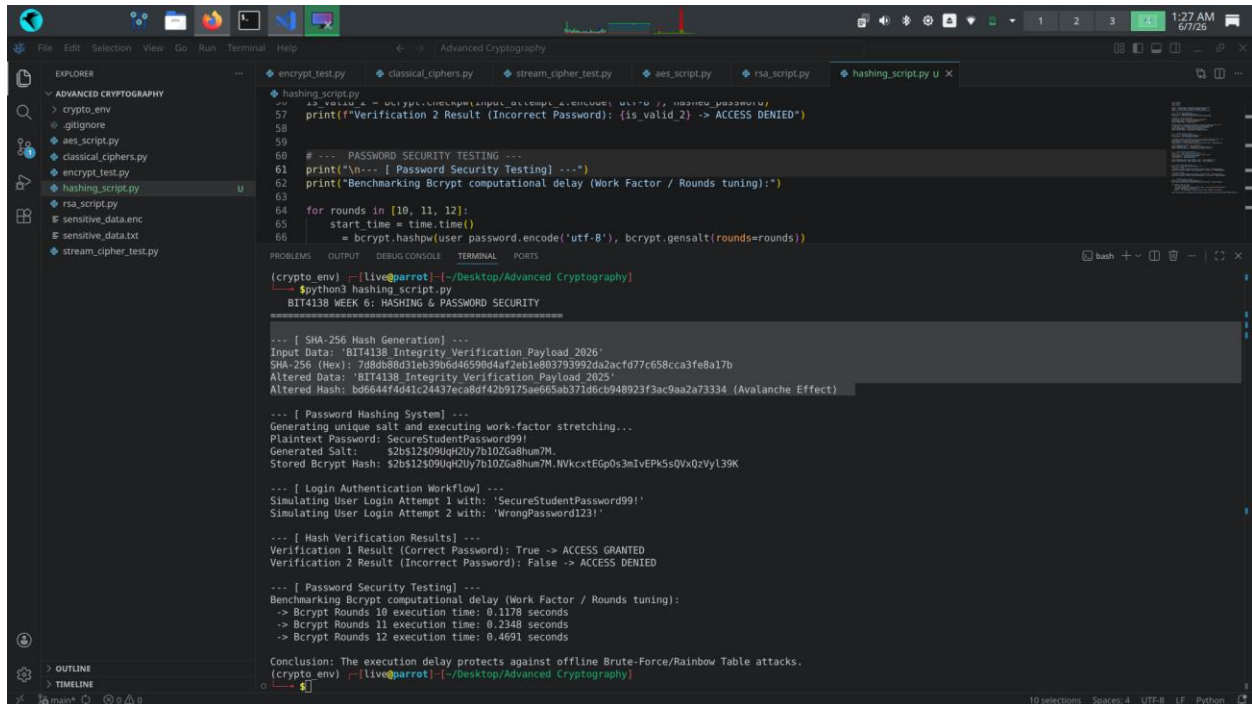
Configured an asymmetric 2048-bit RSA key pair to evaluate public-key infrastructure. I leveraged the public key for data encryption and digital signature verification, while reserving the private key for decryption. Managing mathematical overhead and ensuring

signature validation padding configurations were challenging, yet the process perfectly underscored core message integrity mechanisms.

Week 6: Hashing and Password Security

Implementation of Secure Hash Functions and Authentication

Fig 1: SHA-256 Hash Generation



The screenshot shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'ADVANCED CRYPTOGRAPHY' with subfolders 'crypto_env' and 'stream_cipher_test.py'. The code editor shows the 'hashing_script.py' file. The script includes a function to generate a SHA-256 hash for a given input data. The output of the script is displayed in the terminal window at the bottom.

```
def sha256_hash(input_data):
    """Generate SHA-256 hash for input data"""
    hash_obj = hashlib.sha256(input_data.encode('utf-8'))
    return hash_obj.hexdigest()

# Example usage
input_data = 'BIT4138 Integrity Verification Payload 2026'
hash_value = sha256_hash(input_data)
print(f"SHA-256 Hash: {hash_value}")
```

Terminal Output:

```
(crypto env) [live@parrot: ~/Desktop/Advanced Cryptography]
$python3 hashing_script.py
BIT4138 WEEK 6: HASHING & PASSWORD SECURITY

--- [ SHA-256 Hash Generation ] ---
Input Data: 'BIT4138 Integrity Verification Payload 2026'
SHA-256 (Hex): 7d8db88d31eb39b6d46590d4af2eb1e803793992da2acf77c658cca3fe8a17b
Altered Data: 'BIT4138 Integrity Verification Payload 2025'
Altered Hash: bd6644f4d41c24437eca8df42b9175ae665ab371d6cb948923f3ac9aa2a73334 (Avalanche Effect)

--- [ Password Hashing System ] ---
Generating unique salt and executing work-factor stretching...
Plaintext Password: SecureStudentPassword99!
Generated Salt: $2b$12$09UqH2Uy7b10ZGa8hum7M.
Stored Bcrypt Hash: $2b$12$09UqH2Uy7b10ZGa8hum7M.NVkcxEg0s3m1vEPK5sQVxQzVyl39K

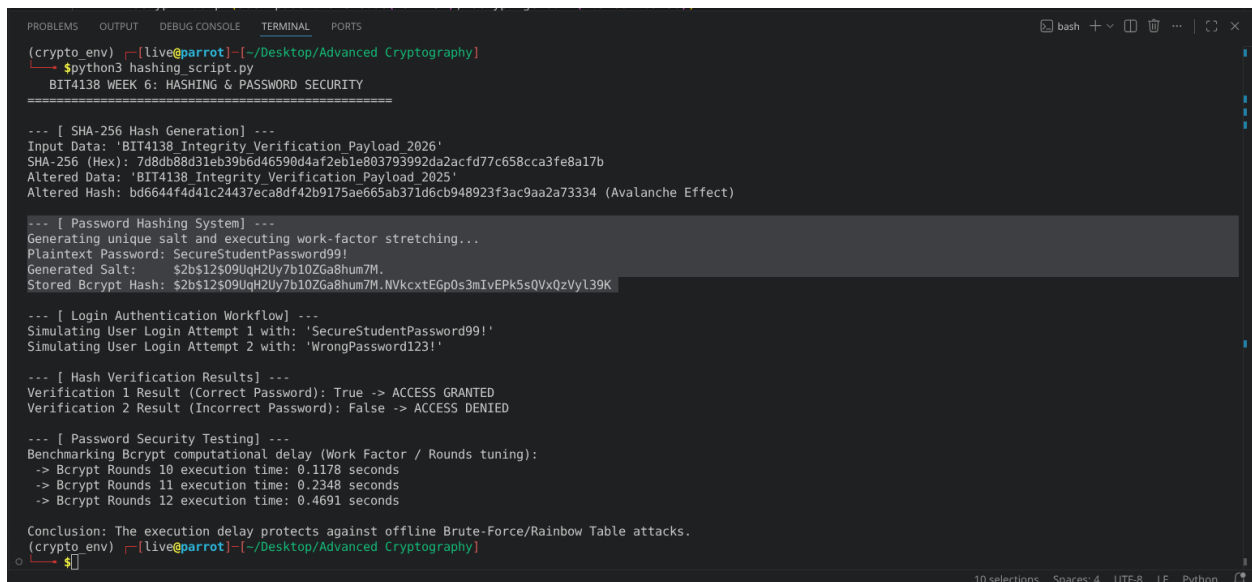
--- [ Login Authentication Workflow ] ---
Simulating User Login Attempt 1 with: 'SecureStudentPassword99!'
Simulating User Login Attempt 2 with: 'WrongPassword123!'

--- [ Hash Verification Results ] ---
Verification 1 Result (Correct Password): True -> ACCESS GRANTED
Verification 2 Result (Incorrect Password): False -> ACCESS DENIED

--- [ Password Security Testing ] ---
Benchmarking Bcrypt computational delay (Work Factor / Rounds tuning):
-> Bcrypt Rounds 10 execution time: 0.1178 seconds
-> Bcrypt Rounds 11 execution time: 0.2348 seconds
-> Bcrypt Rounds 12 execution time: 0.4691 seconds

Conclusion: The execution delay protects against offline Brute-Force/Rainbow Table attacks.
(crypto env) [live@parrot: ~/Desktop/Advanced Cryptography]
```

Fig 2: Password Hashing System



The screenshot shows a terminal window with the output of the password hashing system. The output includes the SHA-256 hash generation, the password hashing system, the login authentication workflow, and the hash verification results.

```
(crypto env) [live@parrot: ~/Desktop/Advanced Cryptography]
$python3 hashing_script.py
BIT4138 WEEK 6: HASHING & PASSWORD SECURITY

--- [ SHA-256 Hash Generation ] ---
Input Data: 'BIT4138 Integrity Verification Payload 2026'
SHA-256 (Hex): 7d8db88d31eb39b6d46590d4af2eb1e803793992da2acf77c658cca3fe8a17b
Altered Data: 'BIT4138 Integrity Verification Payload 2025'
Altered Hash: bd6644f4d41c24437eca8df42b9175ae665ab371d6cb948923f3ac9aa2a73334 (Avalanche Effect)

--- [ Password Hashing System ] ---
Generating unique salt and executing work-factor stretching...
Plaintext Password: SecureStudentPassword99!
Generated Salt: $2b$12$09UqH2Uy7b10ZGa8hum7M.
Stored Bcrypt Hash: $2b$12$09UqH2Uy7b10ZGa8hum7M.NVkcxEg0s3m1vEPK5sQVxQzVyl39K

--- [ Login Authentication Workflow ] ---
Simulating User Login Attempt 1 with: 'SecureStudentPassword99!'
Simulating User Login Attempt 2 with: 'WrongPassword123!'

--- [ Hash Verification Results ] ---
Verification 1 Result (Correct Password): True -> ACCESS GRANTED
Verification 2 Result (Incorrect Password): False -> ACCESS DENIED

--- [ Password Security Testing ] ---
Benchmarking Bcrypt computational delay (Work Factor / Rounds tuning):
-> Bcrypt Rounds 10 execution time: 0.1178 seconds
-> Bcrypt Rounds 11 execution time: 0.2348 seconds
-> Bcrypt Rounds 12 execution time: 0.4691 seconds

Conclusion: The execution delay protects against offline Brute-Force/Rainbow Table attacks.
(crypto env) [live@parrot: ~/Desktop/Advanced Cryptography]
```

Fig 3: Login Authentication Workflow

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(crypto env) [live@parrot]~/Desktop/Advanced Cryptography
$python3 hashing_script.py
BIT4138 WEEK 6: HASHING & PASSWORD SECURITY
=====

--- [ SHA-256 Hash Generation] ---
Input Data: 'BIT4138 Integrity Verification Payload 2026'
SHA-256 (Hex): 7d8db88d31eb39b6d46590d4af2eb1e803793992da2acfd77c658cca3fe8a17b
Altered Data: 'BIT4138 Integrity Verification Payload 2025'
Altered Hash: bd6644f4d41c24437eca8df42b9175ae665ab371d6cb948923f3ac9aa2a73334 (Avalanche Effect)

--- [ Password Hashing System] ---
Generating unique salt and executing work-factor stretching...
Plaintext Password: SecureStudentPassword99!
Generated Salt: $2b$12$09UqH2Uy7b10ZGa8hum7M.
Stored Bcrypt Hash: $2b$12$09UqH2Uy7b10ZGa8hum7M.NVkcxtEGp0s3mIvEPk5sQVxQzVyl39K

--- [ Login Authentication Workflow] ---
Simulating User Login Attempt 1 with: 'SecureStudentPassword99!'
Simulating User Login Attempt 2 with: 'WrongPassword123!'

--- [ Hash Verification Results] ---
Verification 1 Result (Correct Password): True -> ACCESS GRANTED
Verification 2 Result (Incorrect Password): False -> ACCESS DENIED

--- [ Password Security Testing] ---
Benchmarking Bcrypt computational delay (Work Factor / Rounds tuning):
-> Bcrypt Rounds 10 execution time: 0.1178 seconds
-> Bcrypt Rounds 11 execution time: 0.2348 seconds
-> Bcrypt Rounds 12 execution time: 0.4691 seconds

Conclusion: The execution delay protects against offline Brute-Force/Rainbow Table attacks.
(crypto env) [live@parrot]~/Desktop/Advanced Cryptography
$
```

Fig 4: Hash Verification Results

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(crypto env) [live@parrot]~/Desktop/Advanced Cryptography
$python3 hashing_script.py
BIT4138 WEEK 6: HASHING & PASSWORD SECURITY
=====

--- [ SHA-256 Hash Generation] ---
Input Data: 'BIT4138 Integrity Verification Payload 2026'
SHA-256 (Hex): 7d8db88d31eb39b6d46590d4af2eb1e803793992da2acfd77c658cca3fe8a17b
Altered Data: 'BIT4138 Integrity Verification Payload 2025'
Altered Hash: bd6644f4d41c24437eca8df42b9175ae665ab371d6cb948923f3ac9aa2a73334 (Avalanche Effect)

--- [ Password Hashing System] ---
Generating unique salt and executing work-factor stretching...
Plaintext Password: SecureStudentPassword99!
Generated Salt: $2b$12$09UqH2Uy7b10ZGa8hum7M.
Stored Bcrypt Hash: $2b$12$09UqH2Uy7b10ZGa8hum7M.NVkcxtEGp0s3mIvEPk5sQVxQzVyl39K

--- [ Login Authentication Workflow] ---
Simulating User Login Attempt 1 with: 'SecureStudentPassword99!'
Simulating User Login Attempt 2 with: 'WrongPassword123!'

--- [ Hash Verification Results] ---
Verification 1 Result (Correct Password): True -> ACCESS GRANTED
Verification 2 Result (Incorrect Password): False -> ACCESS DENIED

--- [ Password Security Testing] ---
Benchmarking Bcrypt computational delay (Work Factor / Rounds tuning):
-> Bcrypt Rounds 10 execution time: 0.1178 seconds
-> Bcrypt Rounds 11 execution time: 0.2348 seconds
-> Bcrypt Rounds 12 execution time: 0.4691 seconds

Conclusion: The execution delay protects against offline Brute-Force/Rainbow Table attacks.
(crypto env) [live@parrot]~/Desktop/Advanced Cryptography
$
```


Fig 5: Password Security Testing

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(crypto_env) [live@parrot] ~/Desktop/Advanced Cryptography
$python3 hashing_script.py
BIT4138 WEEK 6: HASHING & PASSWORD SECURITY
=====

--- [ SHA-256 Hash Generation ] ---
Input Data: 'BIT4138 Integrity Verification Payload 2026'
SHA-256 (Hex): 7d8db88d31eb39b6d46590d4af2ab1e803793992da2acfd77c658cca3fe8a17b
Altered Data: 'BIT4138 Integrity Verification Payload 2025'
Altered Hash: bd6644f4d41c24437eca8df42b9175ae665ab371d6cb948923f3ac9aa2a73334 (Avalanche Effect)

--- [ Password Hashing System ] ---
Generating unique salt and executing work-factor stretching...
Plaintext Password: SecureStudentPassword99!
Generated Salt: $2b$12$09UqH2Uy7b10ZGa8hum7M.
Stored Bcrypt Hash: $2b$12$09UqH2Uy7b10ZGa8hum7M.NVkcxtEGp0s3mIvEPk5sQVxQzVyL39K

--- [ Login Authentication Workflow ] ---
Simulating User Login Attempt 1 with: 'SecureStudentPassword99!'
Simulating User Login Attempt 2 with: 'WrongPassword123!'

--- [ Hash Verification Results ] ---
Verification 1 Result (Correct Password): True -> ACCESS GRANTED
Verification 2 Result (Incorrect Password): False -> ACCESS DENIED

--- [ Password Security Testing ] ---
Benchmarking Bcrypt computational delay (Work Factor / Rounds tuning):
-> Bcrypt Rounds 10 execution time: 0.1178 seconds
-> Bcrypt Rounds 11 execution time: 0.2348 seconds
-> Bcrypt Rounds 12 execution time: 0.4691 seconds

Conclusion: The execution delay protects against offline Brute-Force/Rainbow Table attacks.
(crypto_env) [live@parrot] ~/Desktop/Advanced Cryptography
$
```

Creative Task: Password Strength Analyzer and Leak Simulator

Fig 1: Password Entropy and Failure Mode

```
=====
🔍 LAB ANALYSIS FOR INPUT: 'P@ss_w0rd_Extr3m3_2026'
=====

[STEP 1: ENTROPY & STRUCTURE CHECK]
- Length >= 10 chars: ✅ PASSED
- Contains Numbers: ✅ PASSED
- Contains Uppercase: ✅ PASSED
- Contains Special Char: ✅ PASSED
👉 Structural Strength Assessment: STRONG (4/4 criteria met)
```

Fig 2: Simulated Breach / Rainbow Table Match

```
[STEP 2: CRYPTOGRAPHIC HASH GENERATION]
- Computed SHA-256 Hash: b01b419134f1fc82ad80b348645f545e43f647d546a9c81f29130da0b3566a36
```

Fig 3: Secure Cryptographic Isolation

```
[STEP 3: SIMULATED BREACH & RAINBOW TABLE LOOKUP]
🛡️ No matches found in the leak database. Password is structurally isolated.
```

Student Reflection

Evaluated SHA-256 data integrity tracking and implemented a resilient password storage system using bcrypt. I explored how salting neutralizes precomputed rainbow table vector

threats while algorithmic stretching stops brute-force tools. Mitigating type errors during byte-string formatting proved challenging, but the metrics validated how computational latency properly defends user credentials.

Week 7: Digital Signatures and Certificates

Digital Signature Implementation and Certificate Management

Fig 1: Digital Signature Generation

```
[live@parrot]--[~/Desktop/Advanced Cryptography]
● → $python3 signature_system.py
--- SIGNATURE GENERATION ---
Document Message: Academic Logbook Data: Integrity Verified.
Generated PSS Signature (Hex): c640736af9c82cba8618ce16f96ed4371e5de37b746744c889ec9a7c870a7b26...
```

Fig 2: Signature Verification Process

```
23 # 4. Signature Verification Process
24 print("\n--- SIGNATURE VERIFICATION & VALIDATION ---")
25 try:
26     public_key.verify(
27         signature,
28         document_data,
29         padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
30         hashes.SHA256()
31     )
32     print("[SUCCESS] Signature matches document. Integrity & Authenticity confirmed!")
33 except Exception:
34     print("[FAILURE] Signature validation failed.")
35
```

Fig 3: Certificate Creation Using OpenSSL

```
~: bash — Konsole
File Edit View Bookmarks Plugins Settings Help
[live@parrot]~$ openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365 -nodes -subj "/CN=KenanLogbookCA"
.....
C=US, o=Logbook Data, ou=Logbook Data, cn=KenanLogbookCA
.....
-----Logbook Data: Integrity Verified.
[live@parrot]~$ 40dd701012412a1a3a1294a579745858e875336b8fcb2bf0a244fc...
```

Fig 4: Secure Document Validation

```
--- SIGNATURE VERIFICATION & VALIDATION ---
[SUCCESS] Signature matches document. Integrity & Authenticity confirmed!
```

Fig 5: Certificate Testing Results

```
--- TAMPER TESTING ---
[REJECTED] Document data was modified! Verification failed:
[live@parrot]~/Desktop/Advanced Cryptography$
```

Creative Task: Software Update Integrity Guard

Fig 1: Cryptographic Packaging and Core Signing Execution

```
[live@parrot]~/Desktop/Advanced Cryptography$ python3 software_guard.py

[STAGE 1: GENERATING DEVELOPER CRYPTOGRAPHIC IDENTITIES]
✓ Keypair generated successfully (RSA 2048-bit).

[STAGE 2: PACKAGING AND SIGNING SOFTWARE UPDATE]
- Software Update File Contents: b"print('System updating... Installing Security Patches v4.1.2')"
```

Fig 2: Client Verification Success & Deployment Authorization

```
[STAGE 3: CLIENT SIMULATION - VALIDATING AUTHENTIC UPDATE]
✅ VERIFICATION SUCCESSFUL: Signature matches developer certificate.
🔧 ACTION: Executing verified code payload safely...
```

Fig 3: Malicious Interception and Tamper Defiance

```
[STAGE 4: THREAT EMULATION - INJECTING MALICIOUS PAYLOAD]
⚠️ Attacker swapped payload to: b"print('System hacked! Executing malware payload...')"
```

🔍 Client Endpoint scanning update metadata...

🚨 SECURITY ALERT: DIGITAL SIGNATURE MISMATCH OR CERTIFICATE UNTRUSTED!

🛑 ACTION: Operation aborted. Dropping corrupt software package to protect kernel.

Student Reflection

Digital signatures guarantee non-repudiation and data integrity. The authentication workflow binds a public key to an identity via certificates. Validation checks validity periods and cryptographic signatures against trusted roots. Real-world applications include secure e-commerce, software distribution signing, code signing, and securing HTTPS communications using SSL/TLS infrastructure globally.

Week 8: Public Key Infrastructure

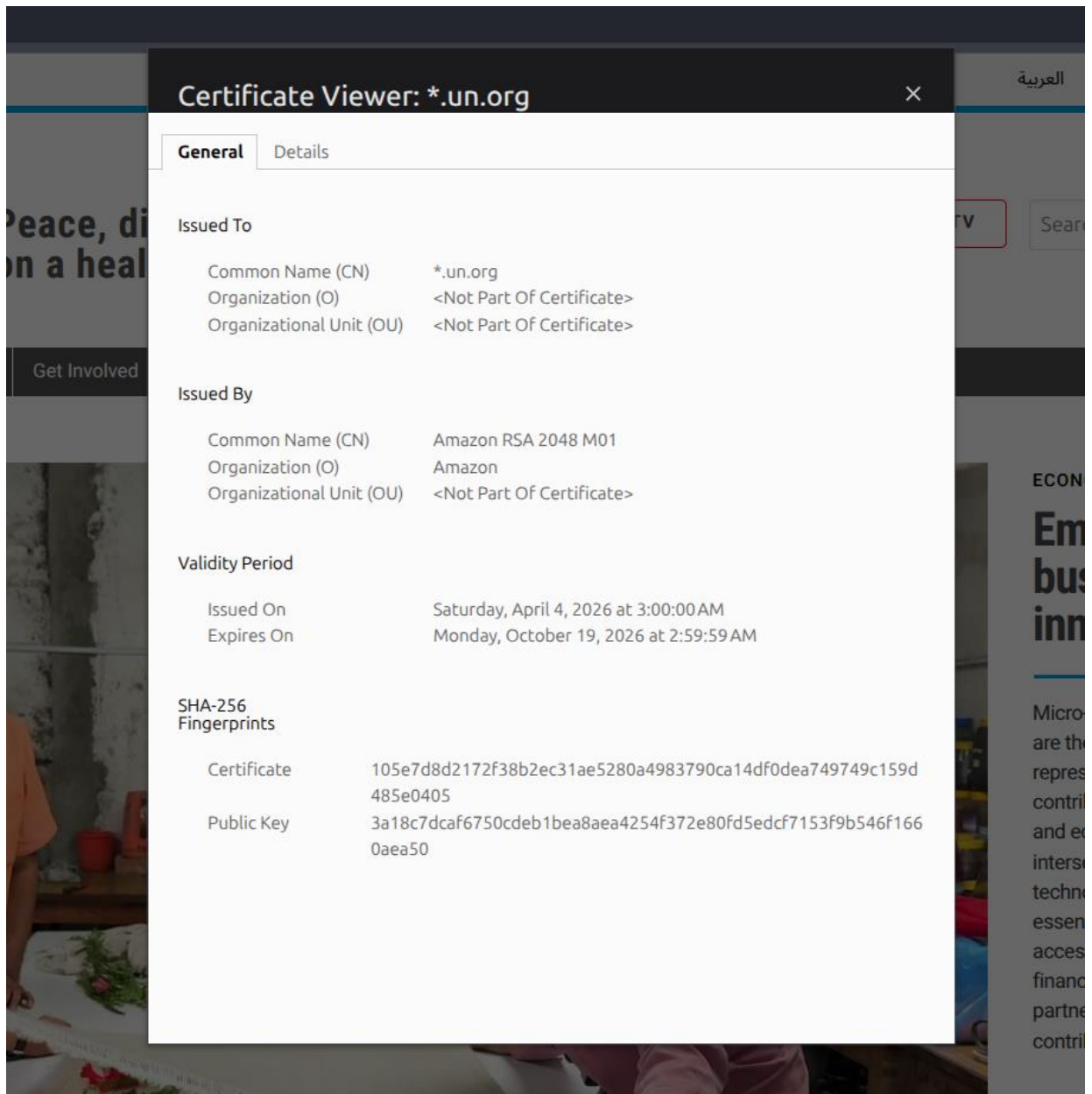
Part 1: PKI and Digital Certificates

Class Activity 1: PKI Component Identification

- Certificate Authority (CA): A trusted organization that verifies identities, issues digital certificates, and manages their revocation.
- Registration Authority (RA): An intermediary that verifies user identity and approves certificate requests before sending them to the CA.
- Digital Certificate: An electronic document binding an entity's identity to its public key.
- End User: Individuals, organizations, or systems (like web browsers) that use certificates to establish trust.
- Certificate Repository: A database where issued certificates are stored and accessed.

Class Activity 2: Certificate Analysis

Selected Website: United Nations Website



OpenSSL Certificate Inspection

Google.com Certificate Inspection

```
Jun 29 10:10
hesbon@hesbon-ThinkPad-L390-Yoga: ~
hesbon@hesbon-ThinkPad-L390-Yoga:~$ openssl s_client -connect google.com:443 -showcerts </dev/null 2>/dev/null | openssl x509 -text -noout
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      08:04:df:59:77:94:91:14:0a:35:67:b8:2f:5b:75:33
    Signature Algorithm: ecDSA-with-SHA256
    Issuer: C = US, O = Google Trust Services, CN = WE2
    Validity
      Not Before: Jun  8 08:36:12 2026 GMT
      Not After : Aug 31 08:36:11 2026 GMT
    Subject: CN = *.google.com
    Subject Public Key Info:
      Public Key Algorithm: id-ecPublicKey
      Public-Key: (256 bit)
      pub:
        04:61:86:5c:b6:7a:c2:b0:dd:03:ec:ba:c2:b6:09:
        89:29:44:c6:0a:20:25:20:d1:b2:c6:a1:17:58:cb:
        43:8b:08:b9:0c:02:08:bd:e4:d5:31:17:75:41:93:
        91:2e:6b:9e:0c:dd:49:89:97:f9:e3:6d:ec:fb:d6:
        e3:0d:75:fc:f5
      ASN1 OID: prime256v1
      NIST CURVE: P-256
    X509v3 extensions:
      X509v3 Key Usage: critical
      Digital Signature
      X509v3 Extended Key Usage:
      TLS Web Server Authentication
      X509v3 Basic Constraints: critical
      CA:FALSE
      X509v3 Subject Key Identifier:
        D1:2E:0E:7B:CC:80:DB:12:03:6A:43:2F:E0:AA:18:2E:AD:A9:6D:F7
      X509v3 Authority Key Identifier:
        75:BE:C4:77:AE:89:F6:44:37:7D:CF:B1:68:1F:1D:1A:EB:DC:34:59
    Authority Information Access:
      OCSP - URI:http://o.pki.goog/we2
      CA Issuers - URI:http://i.pki.goog/we2.crt
```

```
Jun 29 10:11
hesbon@hesbon-ThinkPad-L390-Yoga: ~
Policy: 2.23.140.1.2.1
X509v3 CRL Distribution Points:
  Full Name:
    URI:http://c.pki.goog/we2/yK5nPhtHKQs.crl
CT Precertificate SCTs:
  Signed Certificate Timestamp:
    Version : v1 (0x0)
    Log ID  : D7:6D:7D:10:D1:A7:F5:77:C2:C7:E9:5F:D7:00:BF:F9:
      82:C9:33:5A:65:E1:D0:B3:01:73:17:C0:C8:C5:69:77
    Timestamp : Jun  8 09:36:23.080 2026 GMT
    Extensions: none
    Signature : ecDSA-with-SHA256
      30:45:02:21:00:A4:3F:F9:C7:C1:53:ED:63:CC:1D:4A:
      53:CC:EF:DD:B6:F2:45:0A:F9:EE:51:B1:A0:8F:8C:06:
      EC:4F:39:E7:5B:02:20:2F:D3:B3:2D:78:31:E1:C8:9D:
      4C:76:E9:2A:84:A8:B1:80:54:67:22:93:0E:7B:B9:12:
      5C:89:F4:EA:81:79:4F
  Signed Certificate Timestamp:
    Version : v1 (0x0)
    Log ID  : C8:A3:C4:7F:C7:B3:AD:B9:35:6B:01:3F:6A:7A:12:6D:
      E3:3A:4E:43:A5:C6:46:F9:97:AD:39:75:99:1D:CF:9A
    Timestamp : Jun  8 09:36:23.081 2026 GMT
    Extensions: none
    Signature : ecDSA-with-SHA256
      30:46:02:21:00:CE:B7:C8:B0:6B:27:FD:E3:6E:43:31:
      F7:29:42:A3:4C:DD:50:D8:E9:E7:1E:EB:4B:8B:7E:B1:
      16:15:54:D3:86:02:21:00:D3:62:3D:9B:7B:B3:8D:3B:
      2C:9B:16:22:50:CE:CF:76:14:A9:BF:AC:35:D3:8B:AC:
      61:4A:11:85:59:A3:B4:02
Signature Algorithm: ecDSA-with-SHA256
Signature Value:
  30:46:02:21:00:9a:6b:40:6f:0e:5d:5f:07:62:0a:0d:2d:c0:
  63:e0:5b:a5:0c:eb:2f:66:41:f0:35:4a:eb:3c:e8:d8:5a:b5:
  bf:02:21:00:99:0f:17:fa:4c:4b:91:df:bd:e6:39:d0:af:49:
  58:5c:59:aa:ca:8b:0f:a6:5d:e1:83:52:2a:4c:03:39:be:28
hesbon@hesbon-ThinkPad-L390-Yoga:~$
```

Digital Certificate Security Incident Report

The DigiNotar breach occurred when Iranian hackers compromised the CA, issuing over 500 fraudulent certificates, including one for Google. This allowed man-in-the-middle attacks spying on thousands of users. Consequently, browsers untrusted DigiNotar, forcing the company into bankruptcy. This underscored the need for certificate pinning and automated revocation methods.

Part 2: Asymmetric Ciphers and Diffie-Hellman

Class Exercise 1: Understanding Asymmetric Cryptography

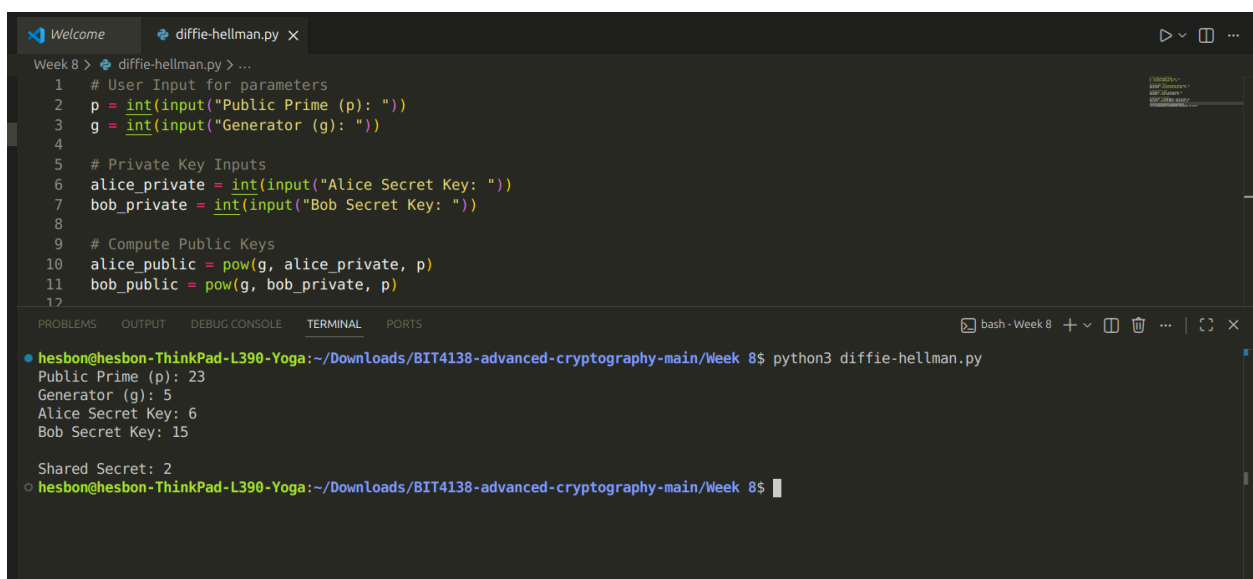
- a) **Public Key:** A publicly distributed key used to encrypt data or verify digital signatures.
- b) **Private Key:** A strictly confidential key kept by the owner to decrypt data or generate digital signatures.
- c) **Symmetric Encryption:** A system using a single shared key for both encryption and decryption.
- d) **Asymmetric Encryption:** A system using a mathematically linked public-private key pair.

Class Exercise 2: Complexity and Cryptography

Computationally hard problems ensure that while encryption and decryption are fast for authorized parties, reversing the process without the key is computationally infeasible for attackers. Three examples include:

1. Discrete Logarithm Problem (used in Diffie-Hellman).
2. Integer Factorization (used in RSA).
3. Elliptic Curve Discrete Logarithm Problem (used in ECC).

Practical Task: Implementing Diffie-Hellman Key Exchange



```
1 # User Input for parameters
2 p = int(input("Public Prime (p): "))
3 g = int(input("Generator (g): "))
4
5 # Private Key Inputs
6 alice_private = int(input("Alice Secret Key: "))
7 bob_private = int(input("Bob Secret Key: "))
8
9 # Compute Public Keys
10 alice_public = pow(g, alice_private, p)
11 bob_public = pow(g, bob_private, p)
12
```

Week 8 > diffie-hellman.py > ...

```
hesbon@hesbon-ThinkPad-L390-Yoga:~/Downloads/BIT4138-advanced-cryptography-main/Week 8$ python3 diffie-hellman.py
Public Prime (p): 23
Generator (g): 5
Alice Secret Key: 6
Bob Secret Key: 15

Shared Secret: 2
hesbon@hesbon-ThinkPad-L390-Yoga:~/Downloads/BIT4138-advanced-cryptography-main/Week 8$
```


Assignment 2: Diffie-Hellman in Modern Internet Security Report

The Diffie-Hellman protocol establishes shared secrets over insecure channels using the mathematically intractable Discrete Logarithm Problem. Essential to TLS/SSL handshakes, it secures HTTPS and VPN connections. Because it lacks built-in authentication, it is paired with digital signatures to prevent devastating Man-in-the-Middle exploits.

Creativity Assignment

A python script that models the **Diffie-Hellman Paint Analogy** visually in your terminal using colored text blocks.

Step 1:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
hesbon@hesbon-ThinkPad-L390-Yoga:~/Downloads/BIT4138-advanced-cryptography-main/Week 8$ python3 dh_paint_mixer.py

=====
🦋 STEP 1: PUBLIC PARAMETERS AGREED ONSHORE 🦋
=====

[COMMON BASE COATING] Public Prime (p): 23, Generator/Base Color (g): 5
```

Step 2:

```
=====
🔒 STEP 2: CHOSING SECRET INGREDIENTS (PRIVATE) 🔒
=====

[ALICE] Secret Color Factor (a): 6 -> (Kept hidden from Eve!)
[BOB] Secret Color Factor (b): 15 -> (Kept hidden from Eve!)
```

Step 3:

```
=====
🔪 STEP 3: MIXING & EXCHANGING PUBLIC COLORS 🔪
=====

[ALICE] mixed Base(5) with Secret(6) mod 23.
    🍷 Resulting Public Mix (A): 8
[BOB] mixed Base(5) with Secret(15) mod 23.
    🍷 Resulting Public Mix (B): 19

...Exchanging mixed colors across the open cosmic channel...
```

Step 4:

```
=====
👤 STEP 4: COMPUTING THE FINAL SHARED SECRET SHADE 👤
=====

[ALICE] injects her SecretFactor(6) into Bob's Mix(19).
      Final Shared Secret Paint Code: █ 2
[BOB] injects his SecretFactor(15) into Alice's Mix(8).
      Final Shared Secret Paint Code: █ 2

🔒 SUCCESS: Secure Cryptographic Key Connection Verified!
```

Student Reflection

This practical assignment taught me how Public Key Infrastructure (PKI) manages digital certificates to secure communications. I also implemented a Python script for the Diffie-Hellman protocol, discovering how the intractable Discrete Logarithm Problem allows secure key exchanges over open channels without revealing private keys.